Linux Systems and Performance

PART 3: Memory Management

Week 7: Virtual Memory (Application View)

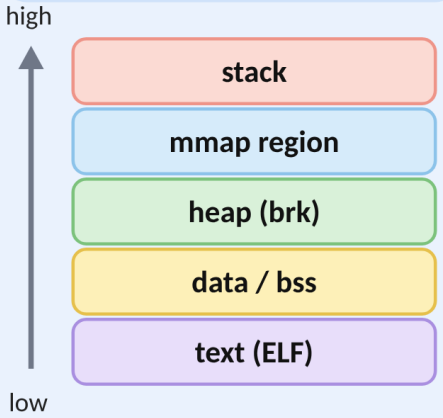
Sungyong Park, Ph.D.
Data-Intensive AI Computing and Systems (DISCOS) Laboratory
Department of Computer Science and Engineering, Sogang University
URL: https://discos.sogang.ac.kr
E-mail: parksy@sogang.ac.kr

Roadmap of Part 3

This week

7 Virtual memory (application view)

How does a process see and use its memory?



- Process address space and ELF mappings
- VMAs, brk() vs mmap()
- malloc() and the glibc allocator

8 Address translation (kernel view)

How does a virtual address become a physical page?

Virtual address
from the CPU

↓

Page table walk
TLB lookup first

↓

Physical page
DRAM

- Page tables and the TLB
- Page faults (minor and major)
- Buddy and slab allocators

9 Memory pressure and page reclaim

How does Linux recover pages under pressure?

Memory pressure
free pages run low

↓

Reclaim pages
kswapd / direct

↓

Free pages restored
allocation succeeds

- Watermarks, kswapd vs direct reclaim
- LRU, swap and zram
- Observing pressure (PSI, cgroup)

10 Memory shortage, recovery, and OOM

What happens when reclaim cannot keep up?

Reclaim cannot keep up
severe pressure

↓

OOM killer / LMK
pick a victim

↓

Memory recovered
system keeps running

- OOM killer: who dies, and why
- LMK, PSI and user-space killers
- Finding the cause: leaks, fragmentation

PART 3: Memory Management

- **Week 7**

- **Process Virtual Address Space (VAS)**
  - Where does a program's memory live?
- **Dynamic VAS Management**
  - How do *brk()* and *mmap()* reshape the address space at runtime?
- **Inside the Allocator**
  - How does ptmalloc manage memory, and what are the performance-footprint trade-offs?

Recap: Virtual Address Space (VAS)

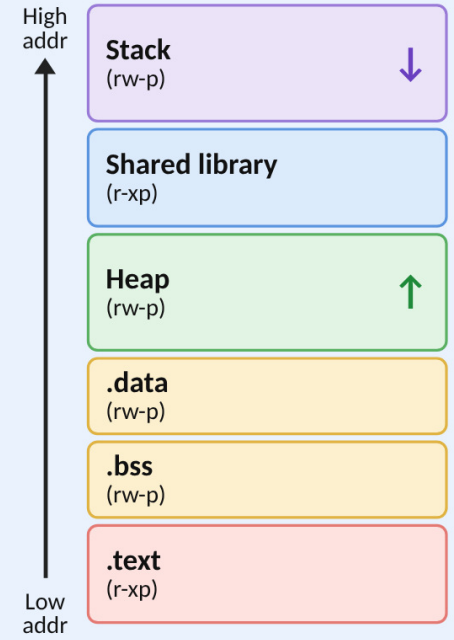
Key Points

- **Processes use virtual addresses**
  Each instruction fetch, load, and store is interpreted in the process address space.
- **Printed pointers are virtual**
  &main, &global, &local and malloc() results are not physical DRAM addresses.
- **Regions have distinct roles**
  .text, .data/.bss, heap, shared libraries, and stack sit at different ranges.
- **The MMU translates**
  OS-managed page tables map each virtual address; a missing mapping raises a page fault.

Recap: where does a virtual address come from?

Every address a program prints lives in its own virtual address space.

1 Process view



Every access uses an address **in that process's** VAS.

2 Printed addresses are virtual

```c
int global = 10;
int main(void) {
  int local = 20;
  int *p = malloc(1024);
  printf("&main   = %p\n", main);
  printf("&global = %p\n", &global);
  printf("&local  = %p\n", &local);
  printf("p       = %p\n", p);
}
```

```
&main   = 0x55ed2b904189
&global = 0x55ed2b907010
&local  = 0x7ffdbadd0e3c
p       = 0x55ed33d1b2a0
```

.text
.data
stack
heap

Not physical DRAM addresses: they are virtual addresses.

Example output: Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18; addresses vary (ASLR).

3 To physical memory

Virtual address
(e.g., 0x7ffdbadd0e3c)

▼

MMU
uses OS-managed page tables

▼

Physical address
(e.g., 0x000012345000)

Mapping not present
→ page fault
Details in Week 8.

Key Takeaway

A running process only sees virtual addresses; the MMU and page tables translate them to physical memory, and a missing mapping triggers a page fault.

ELF Overview: From Objects to One ELF

Key Points

- **Inputs: objects with sections**
  A **section** is a named block of one kind of data: **.text**, **.rodata**, **.data**, **.bss**. Every .o brings its own.
- **Linker: merge into one ELF**
  Same-named sections of all inputs are combined into one. The startup code (crt*.o) is merged too.
- **Shared libs are only recorded**
  libc.so is not copied into the ELF; the linker records the dependency, and ld.so loads it at run time.
- **ELF = headers + sections**
  The **SHT** lists all the sections (the linker's view). The segments and the PHT come next.

From source files to one ELF

Each source becomes an object file with its own sections; the linker merges them into a single executable.

1 Inputs: many objects

each .o has its own sections

main.o
- `.text` code
- `.rodata` constants
- `.data` init. globals
- `.bss` zero globals

util.o
- `.text` code
- `.data` init. globals

crt*.o startup code
- `.text` `.data` `.init` `.fini`

libc.so shared library
not copied in: the linker only records the dependency (symbols, interpreter)

→ **Linker** merges sections, resolves symbols, assigns addresses

2 Output: one ELF file

a.out

- **ELF Header** entry point, where the tables are
- **PHT** program header table (details later)
- **Sections** same-named sections merged
  `.text` `.rodata` `.data` `.bss`
  `.init` `.fini` `.plt` `.got` `.dynamic`
  `.interp`
  `.symtab` `.strtab`
- **SHT** section header table: lists all the sections

colored = merged from the inputs **gray** = added by the linker

Example: gcc -c main.c util.c; gcc main.o util.o -o a.out (Ubuntu 24.04, glibc 2.39, x86-64).

Key Takeaway

The linker merges the sections of all input objects into one ELF and lists them in the SHT.

ELF Overview: Sections and Segments

Key Points

- **Section vs. segment**
  A **section** is the linker's view; a **segment** is the loader's view of the same bytes.
- **Same permissions together**
  A **segment** = one or more sections with the **same permissions** (R, R E, RW).
- **The PHT lists the segments**
  The linker writes the **PHT**: one **PT_LOAD** entry per segment to map.
- **Loader: uses only the PHT**
  The kernel maps each **PT_LOAD segment** using the **PHT**. The SHT is not needed at run time.

From sections to segments

The linker groups sections with the same permissions into segments; the PHT lists them for the loader.

1 Sections — the linker's view (listed in the SHT)		2 Segments — the loader's view (listed in the PHT)
ELF Header entry point, where the tables are		
PHT describes the segments		
`.interp` `.dynsym` `.dynstr` `.rela.dyn` `.rela.plt`	→	**LOAD 1 R** ELF header, PHT and read-only metadata • **ELF Header** and **PHT** • read-only sections: `.interp .dynsym .dynstr .rela.dyn .rela.plt`
`.init` `.plt` `.text` `.fini`	→	**LOAD 2 R E** code
`.rodata` `.eh_frame`	→	**LOAD 3 R** read-only data
`.init_array` `.dynamic` `.got` `.data` `.bss`	→	**LOAD 4 RW** writable data
`.symtab` `.strtab`		in no segment: **not loaded** (symbols and strings for tools)
SHT describes all the sections		**Loader** reads the **PHT** and maps the segments into memory

Key Takeaway

The linker groups sections into segments and writes the PHT; the loader needs only the PHT, not the SHT.

PT_LOAD Details: Sizes and Addresses

Key Points

- **Each PT_LOAD describes one mapping**
  Offset (file), **VirtAddr** (ELF VA), **FileSiz**, **MemSiz** and **Flg** (r/w/x).
- **MemSiz > FileSiz: zero-fill**
  .bss has no bytes in the file. The kernel supplies **zeroed** memory for **MemSiz − FileSiz**.
- **Section VA from its offset**
  Section VA = segment VA + (section offset − segment offset).
- **ELF VA is not the runtime address**
  ELF VA starts near 0. **Runtime = load bias + ELF VA**; the bias is random with PIE/ASLR.
- **Initially one VMA each**
  Each **PT_LOAD** gets a **VMA** (r/w/x from Flg); it can **split later** (RELRO).

What does a PT_LOAD entry tell the loader?

Each entry gives the file range, the address, the sizes and the permissions of one mapping.

1 A PT_LOAD entry
one mapping request to the kernel

File → memory

MemSiz = FileSiz + zero-filled tail (**.bss**)

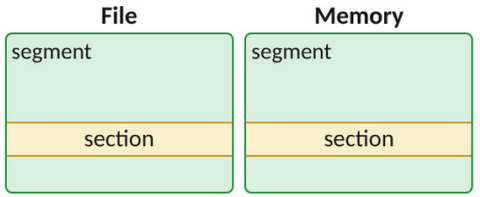
Offset: where the bytes are in the **file**
VirtAddr: the **ELF virtual address** to map at
FileSiz: bytes stored in the file
MemSiz: bytes needed in memory
Flg: permissions → the VMA's r/w/x

What is a VMA?
The kernel's **vm_area_struct**: a contiguous virtual range with the same **permissions** and the same **backing** (a **file**, or **anonymous** zero-filled pages).

2 Address of a section
ELF virtual address inside its segment

Section VA = segment VA + (section offset − segment offset)
The file and the memory image keep the **same distance** from the start of the segment.

Why it works

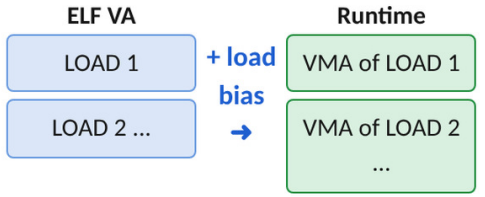


Same **offset inside the segment** in both: the section sits 50 units below the segment start in the file and in memory.

Later: compute it with a **real** a.out.

3 ELF VA vs. runtime address
the loader adds a load bias

The ELF stores addresses **starting near 0**. At execve the kernel picks a **load bias**: **runtime address = load bias + ELF VA**



All segments move together by the **same bias**.

With **PIE** and **ASLR** the bias is random on every run → explained in a later slide.

Key Takeaway

Each PT_LOAD gives offset, VirtAddr, FileSiz, MemSiz and flags; the loader maps it at load bias + VirtAddr and zero-fills MemSiz − FileSiz.

Checking PT_LOAD with readelf

Key Points

- **readelf reads the ELF file**
  No need to run the program: **-h** header, **-l** PHT, **-S** SHT.
- **readelf -l: the segments**
  Shows the **PT_LOAD** entries the loader reads: offset, VirtAddr, sizes, flags.
- **readelf -S: the sections**
  Shows each section's **address** and **file offset** (from the SHT).
- **Formula check**
  0x3db0 + (0x3000 − 0x2db0) = 0x4000, the Address that readelf -S prints for .data.
- **Runtime address**
  0x555555554000 + 0x4010 = 0x555555558010, inside the rw-p VMA in /proc/<pid>/maps.

Checking PT_LOAD with readelf

Real values from one build: segments, sections and the runtime address fit the formulas.

1 PT_LOAD entries

readelf -l: the loader's view

```
$ readelf -lW a.out
Offset VirtAddr FileSiz MemSiz Flg
0x0000 0x0000   0x660   0x660  R
0x1000 0x1000   0x1dd   0x1dd  R E
0x2000 0x2000   0x114   0x114  R
0x2db0 0x3db0   0x268   0x270  RW
```

LOAD 4 (RW) is the data segment.
Offset 0x2db0 → VirtAddr 0x3db0.
MemSiz 0x270 > FileSiz 0x268: 8 bytes of **.bss** are zero-filled.

2 Section addresses

readelf -S and nm

```
$ readelf -SW a.out
Name  Address Off    Size
.data 0x4000  0x3000 0x18
.bss  0x4018  0x3018 0x08
$ nm a.out
0x4010 D g_init
```

Check the formula for .data

```
= 0x3db0 + (0x3000 − 0x2db0)
= 0x3db0 + 0x250
= 0x4000 ✓ (matches Address)
```

g_init 0x4010 lies inside .data (0x4000–0x4018).

3 Runtime address

ASLR off: setarch -R

```
ELF VA      = 0x4010
load bias   = 0x555555554000
runtime     = 0x555555558010
```

```
$ cat /proc/<pid>/maps
...
555555558000-555555559000 rw-p 00003000 ... a.out
```

The runtime address of **g_init** lies in the **rw-p** VMA: the mapping of **LOAD 4** (file offset 0x3000).

`readelf -h`	`readelf -l`	`readelf -S`	`readelf -s / nm`
ELF header	program headers (PHT)	section headers (SHT)	symbols and addresses

Example output: Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18; load bias shown with ASLR off (setarch -R).

Key Takeaway

readelf shows the numbers behind the concepts: PT_LOAD fields, section addresses, and load bias + ELF VA = the runtime address in maps.

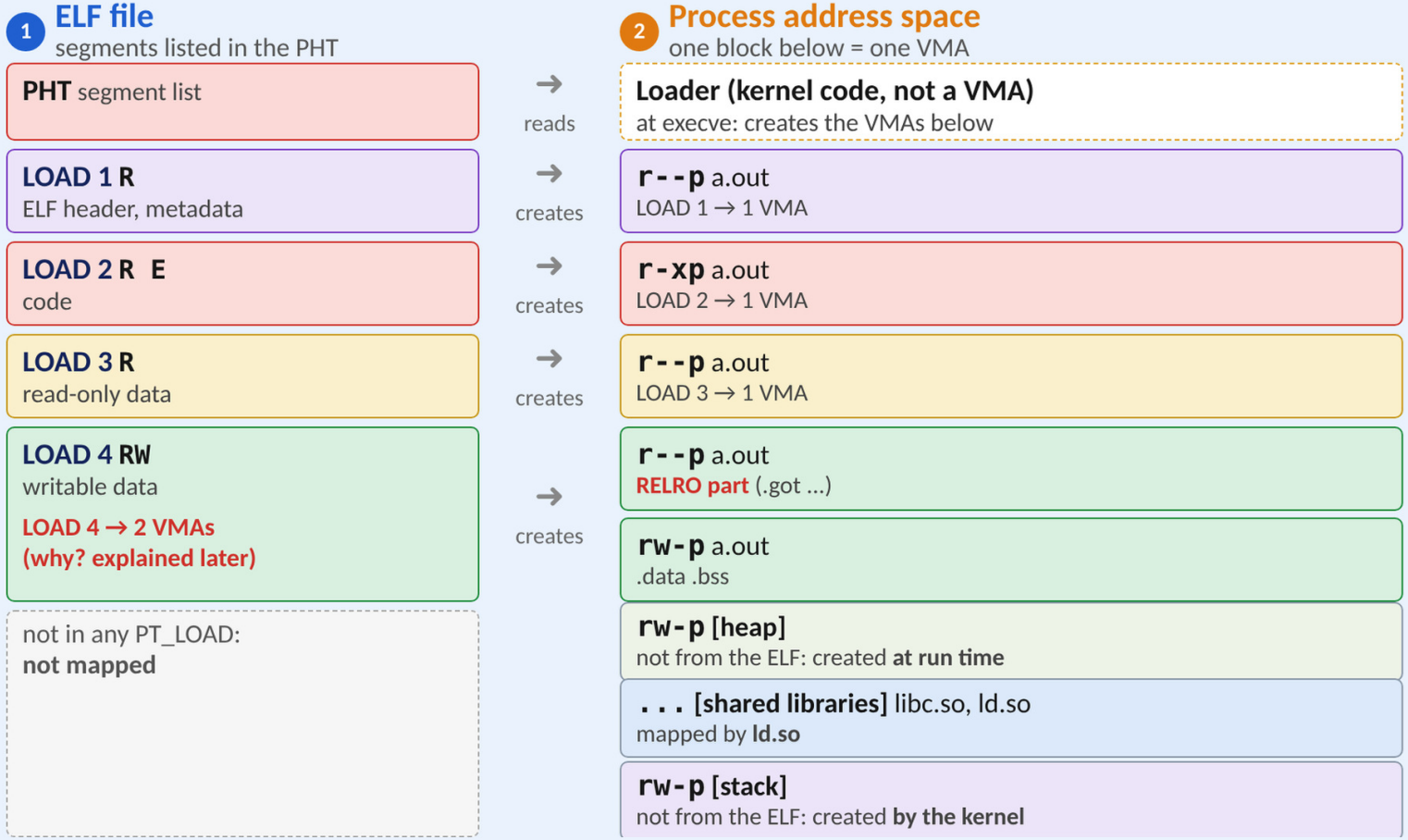
Mapping PT_LOAD to Process VAS

Key Points

- **Loader: reads the PHT**
  At execve, the kernel reads the **PHT** and maps each **PT_LOAD** at its address.
- **Each mapping is a VMA**
  A **VMA** (vm_area_struct) is a contiguous virtual range the **kernel** manages.
- **Not everything is a segment**
  [stack]: kernel, at execve. **[heap]**: run time. **[shared libraries]**: **ld.so**.
- **Mapping is lazy**
  Mapping only sets up VMAs; file pages load on first access (**demand paging**, Week 8).

How does the loader map segments into memory?

The kernel reads the PHT and maps each PT_LOAD segment; each initial mapping is a VMA (it may be split later).



Key Takeaway

The loader maps each PT_LOAD segment as an initial VMA; later changes (e.g., RELRO) can split it, and the kernel manages the address space as VMAs.

Why One Segment Becomes Several VMAs

Key Points

- **VMA = same permissions + backing**
  One **vm_area_struct** covers a range with a single set of **permissions** and one **backing** (file or anonymous).
- **Permissions change: split**
  If permissions change inside a segment, it becomes **several VMAs**.
- **Why RELRO?**
  ld.so must write .got at startup. Afterwards it makes it **read-only** so a memory bug cannot hijack the program.
- **Backing differs: separate VMA**
  File pages and zero-filled **anonymous** pages cannot share a VMA: .bss beyond the file gets its **own VMA**.

Why can one segment become several VMAs?

A VMA has one set of permissions and one backing, so different permissions or backing mean separate VMAs.

1 LOAD 4 (RW): one segment, step by step

what the kernel and ld.so do

1 execve: map LOAD 4

The kernel maps the whole segment **RW**.

`rw-p` .init_array .dynamic .got .data .bss

▼

2 ld.so relocates

ld.so **writes** final addresses into .got, .init_array and .dynamic.

▼

3 mprotect: RELRO

After relocation they never change, so ld.so calls `mprotect` on that part: **read-only**.

`r--p` RELRO: .init_array .dynamic .got

`rw-p` .data .bss

2 Why more VMAs than segments?

a split, or a separate mapping

RELRO (RELocation Read-Only)

Different permissions inside one segment → the kernel **splits** it into two VMAs. **Why?** A writable .got could be overwritten by a memory bug, so the program could be **hijacked**.

Backing: where the pages come from

File-backed
pages read from a.out or libc.so (.text, .rodata, .data)

Anonymous
no file: zero-filled pages (heap, stack, .bss)

A VMA has **one** backing, so file and anonymous pages are always in **different VMAs**.

.bss: no bytes in the file, so it is **anonymous**. If it extends beyond the last file page, the loader creates a **separate anonymous VMA** for the rest (not a split). Here it fits, so no extra VMA.

At run time: `mmap` creates new VMAs (e.g. malloc, shared libraries); `mprotect` and `munmap` can split or merge VMAs.

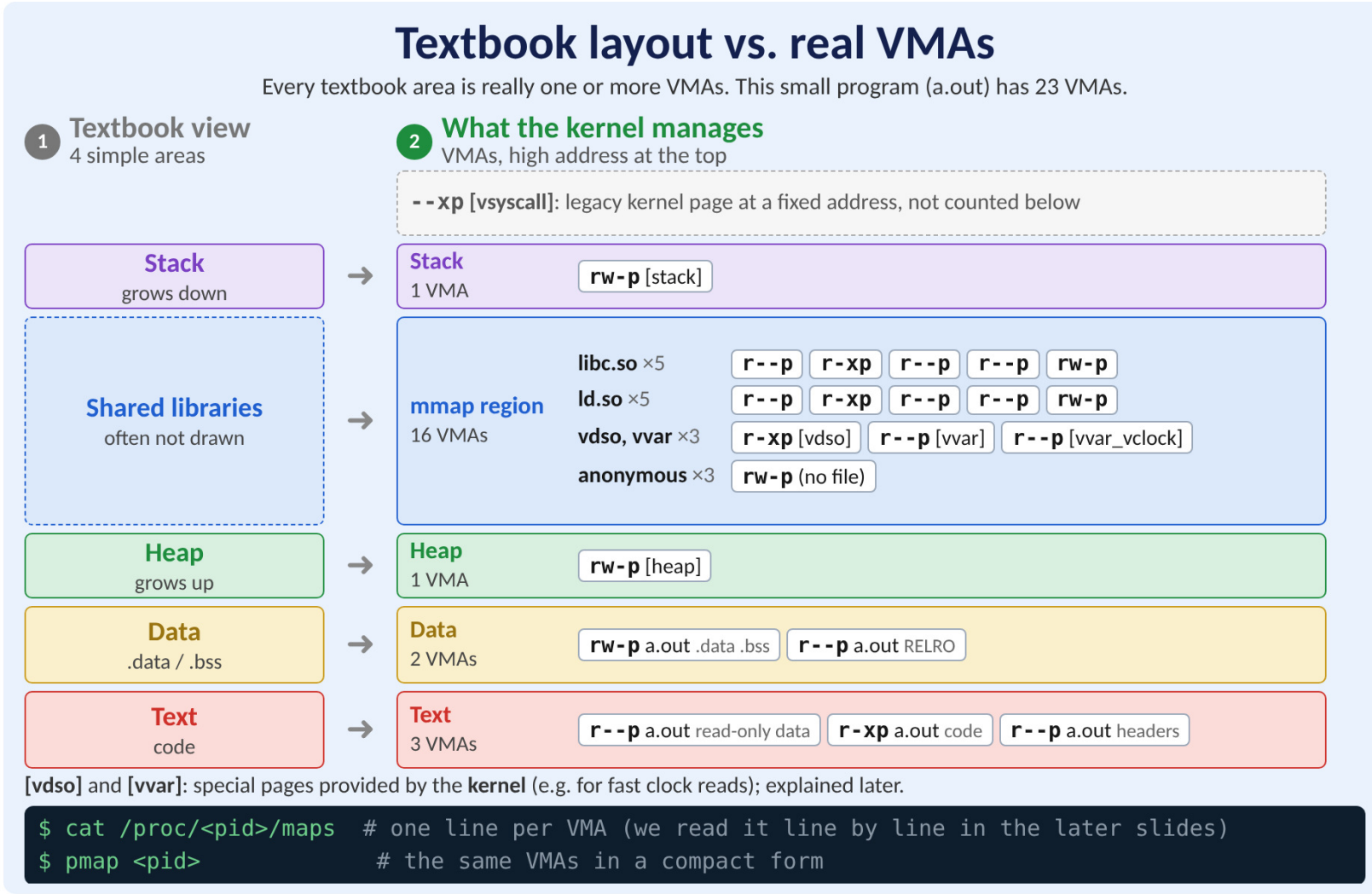
Key Takeaway

One segment becomes several VMAs by a permission change (RELRO) or anonymous backing (.bss).

Textbook Layout vs. Real VMAs

Key Points

- **Textbook view**
  Text, data, heap, stack: a simplified picture, as if each area were one contiguous block.
- **Kernel's view**
  The kernel manages memory as **VMAs**. A new VMA starts wherever **permissions or backing** change.
- **Text and data are split**
  Text = **r--p + r-xp + r--p** (headers, code, constants). Data = **r--p (RELRO) + rw-p**.
- **Not in the textbook**
  Shared libraries (libc.so, ld.so) sit in the **mmap region**, with several VMAs each; **vdso/vvar** are kernel-provided pages.
- **See it yourself**
  /proc/<pid>/maps prints one line per VMA.



Key Takeaway

Text, data, heap and stack are conceptual groups; the kernel actually manages VMAs: here, 23 of them.

Why Do Addresses Change? PIE and ASLR

Key Points

- **PIE: position-independent code**
  The ELF has **no fixed load address**; it can be loaded at **different virtual addresses**.
- **Random load bias at execve**
  Runtime address = load bias + ELF VA; the kernel picks the bias.
- **ASLR randomizes regions**
  The **PIE executable**, **libraries**, **heap**, **stack** and **VDSO/VVAR** move on each new process.
- **Turn it off only to experiment**
  setarch -R gives reproducible addresses for labs, **not** for production.

Why do addresses change on every run?

PIE makes the binary relocatable; ASLR randomizes where everything lands.

1 PIE: Position-Independent Executable

the binary can be loaded at any address

```
$ readelf -h ./a.out
  Type:  DYN (Position-Independent Executable)
  Entry point address:  0x1080
```

The ELF virtual address starts at **0**. At execve() the kernel adds a **random load bias**:
runtime address = load bias + ELF VA

3 Same program, different addresses

values from two runs of a.out

Region	Run 1	Run 2
PIE exec	0x55702a2a1179	0x55ade12af179
Heap	0x55702ac462a0	0x55ade23ad2a0
mmap (libc)	0x7f2e36e60100	0x7fe441860100
Stack	0x7ffe0854bc1c	0x7fff3547d5ac

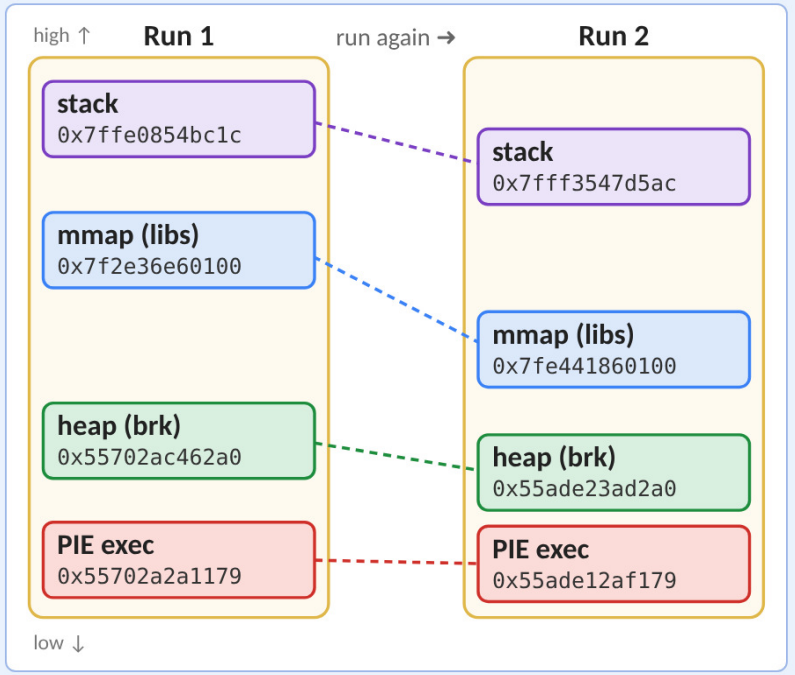
Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18.

```
# Disable ASLR for this process
$ setarch $(uname -m) -R ./a.out
```

Disable ASLR only for controlled experiments, never in production.

2 ASLR: Address Space Layout Randomization

same layout order, new addresses on each run



ASLR randomizes: the **PIE executable**, **shared libraries**, **heap (brk)**, **stack** and **VDSO/VVAR**. The order stays the same; the **gaps** change.

Key Takeaway

The same binary is loaded at different virtual addresses on each run because of PIE and ASLR; disable ASLR only for controlled experiments.

Observing the Address Space

Key Points

- **maps lists current mappings**
  /proc/<pid>/maps shows the virtual-memory mappings of a running process.
- **One line = one VMA**
  Each line is a contiguous address range with the same properties: a **VMA**.
- **Objects split into several VMAs**
  maps is sorted from low to high address; a.out and libraries split into several lines when **permissions** differ.
- **Stack and heap: one VMA each**
  In this example both appear as **a single line**.

What does /proc/<pid>/maps show?

One line of output = one VMA of the process.

1 C program

```
#include ...

int x;
int y = 10;

int main(void)
{
  int a;
  int *p;
  p = malloc(10);
  *p = 20;
  x = y + *p;
  printf("%d",
    getpid());
  while (1);
}
```

2 /proc/<pid>/maps (excerpt)

```
562b6a170000-562b6a171000 r--p a.out
562b6a171000-562b6a172000 r-xp a.out
562b6a172000-562b6a173000 r--p a.out
562b6a173000-562b6a174000 r--p a.out
562b6a174000-562b6a175000 rw-p a.out
562b9c2fe000-562b9c31f000 rw-p [heap]
7f24e2000000-7f24e2028000 r--p libc.so.6
7f24e2028000-7f24e21b1000 r-xp libc.so.6
7f24e2206000-7f24e2213000 rw-p (anon)
7f24e2229000-7f24e222d000 r--p [vvar]
7f24e222d000-7f24e222f000 r--p [vvar_vclock]
7f24e222f000-7f24e2231000 r-xp [vdso]
7f24e2231000-7f24e2232000 r--p ld-linux.so
7f24e2232000-7f24e225d000 r-xp ld-linux.so
7ffd0c618000-7ffd0c63e000 rw-p [stack]
```

3 Process VAS
low → high address

- **a.out** · 5 VMAs
- **[heap]** · 1 VMA
- **libc.so.6** · 5 VMAs
- **anonymous** · 3 VMAs
- **[vvar] [vdso]** · 3 VMAs
- **ld-linux** · 5 VMAs
- **[stack]** · 1 VMA

Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18; offset/device/inode omitted; addresses vary (ASLR).

Why [vdso] and [vvar]? A system call just to read the **current time** is costly. So the kernel maps **[vvar]** (kernel-updated data) and **[vdso]** (tiny code that reads it): **clock_gettime()** becomes a plain memory read.

Key Takeaway

/proc/<pid>/maps shows one line per VMA; an object is split into several VMAs when permissions differ.

Reading One Mapping

Key Points

- **Address range**
  Start – end of the VMA; the end address is **exclusive**.
- **Permissions**
  r, w, x plus **p** (**private**, not shared) or **s** (shared).
- **File offset, device, inode**
  Identify **which file** the VMA maps and **where** in the file it starts.
- **Pathname**
  File path, or a special name such as **[heap]**, **[stack]**, **[vdso]** for anonymous VMAs.

How do you read one mapping line?

Six fields describe every VMA listed in /proc/<pid>/maps.

```
562b6a170000-562b6a171000 r--p 00000000 fe:00 894325 /home/discos/a.out
562b6a171000-562b6a172000 r-xp 00001000 fe:00 894325 /home/discos/a.out
562b6a172000-562b6a173000 r--p 00002000 fe:00 894325 /home/discos/a.out
562b6a173000-562b6a174000 r--p 00002000 fe:00 894325 /home/discos/a.out
562b6a174000-562b6a175000 rw-p 00003000 fe:00 894325 /home/discos/a.out
```

Address range

Start – end address of this VMA. The end address is **exclusive**: the byte at **end** is **not** in this VMA.

```
[0x562b6a170000,
0x562b6a171000)
```

size = end − start = 0x1000 (4 KB)

Permissions

Access rights of the VMA.

```
r read
w write
x execute
p private
s shared
```

p = private: **not shared**; changes stay in this process.
s = shared with other processes.

File offset

Offset in the mapped file where the VMA starts (bytes).

```
00000000
```

Different VMAs of one file can have different offsets. Anonymous VMAs: **0**

Device

Device ID of the filesystem holding the file.

```
fe:00
```

Major:minor number in hexadecimal. Anonymous VMAs: **00:00**

Inode

Inode number of the mapped file.

```
894325
```

Same inode and pathname = same file. Anonymous VMAs: **0**

Pathname

Path of the mapped file.

```
/home/discos/a.out
```

Anonymous VMAs: **blank**, or a special name: [heap], [stack], [vdso], [vvar].

Example output: Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18; addresses vary (ASLR). Same run as the previous slide.

Key Takeaway

Each line of /proc/<pid>/maps is one VMA: address range, permissions, file offset, device, inode, and pathname tell you what it is and where it comes from.

How the Kernel Manages VMAs

Key Points

- **mm_struct holds the VMAs**
  Each process has one **mm_struct**. It indexes all of the process's **vm_area_structs** by address.
- **maps columns = struct fields**
  vm_start/vm_end = address range, **vm_flags** = perms, **vm_pgoff** = file offset, **vm_file** = pathname and inode.
- **Linux 6.1+: Maple Tree**
  A **Maple Tree** (**mm_mt**): **many ranges per node** and lock-free **RCU** reads.
- **Linux 5.x and earlier**
  A **red-black tree** (mm_rb), **one key per node**, plus a **linked list** (mmap) to walk all VMAs.

How does the kernel manage VMAs?

Each maps line is a vm_area_struct; the process's mm_struct indexes them by address (the index changed in Linux 6.1).

1 One maps line = one VMA

stored as a vm_area_struct

```
562b6a171000-562b6a172000 r-xp a.out
562b9c2fe000-562b9c31f000 rw-p [heap]
7f24e2028000-7f24e21b1000 r-xp libc.so
```

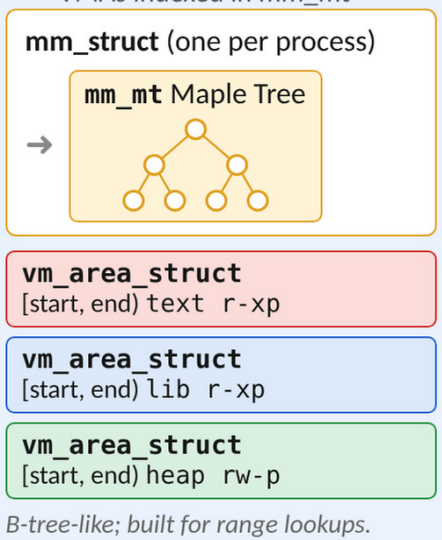
▼ each line is a **vm_area_struct**

maps column	field
address range	`vm_start, vm_end`
perms (r/w/x, p/s)	`vm_flags`
file offset	`vm_pgoff`
pathname, inode	`vm_file`

Anonymous VMA: vm_file is NULL (no pathname). Offset, device and inode columns are omitted above.

2 Linux 6.1+: Maple Tree

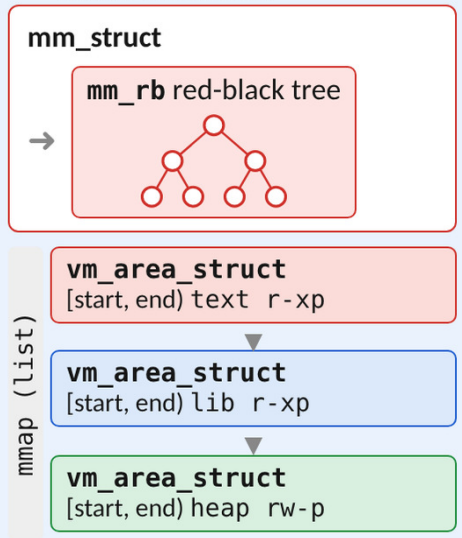
VMAs indexed in mm_mt



B-tree-like; built for range lookups.

3 Linux 5.x and earlier

rbtree + linked list



Tree: lookup. List: in-order walk.

	rbtree (5.x)	Maple Tree (6.1+)
Node	1 key per node	**many ranges** per node
Memory access	pointer chasing	**cache-friendly**
Concurrent reads	need a lock	**lock-free** (RCU)
Walk all VMAs	separate linked list	the tree itself

Example output: Ubuntu 24.04, glibc 2.39, x86-64; addresses vary (ASLR). Paths shortened.

Key Takeaway

Each maps line is one vm_area_struct; the process's mm_struct finds them by address with a Maple Tree (Linux 6.1+) or an rbtree + linked list (earlier).

How Can VMA Be Used?

Key Points

- **Find the VMA first**
  On a page fault, the kernel **searches the VMA** that contains the faulting address.
- **No VMA means SIGSEGV**
  An address outside every VMA is **invalid** and the process receives **SIGSEGV**.
- **vm_flags check permission**
  A disallowed read, write, or execute is a **protection fault**: SIGSEGV; otherwise handling continues.
- **The VMA says how to back the page**
  File-backed, **anonymous**, **BSS**, or **stack** mappings each provide the info needed to resolve the fault.

How does a page fault use the VMA?

The VMA validates the address, checks permission, and tells the kernel how to back the page.

2 Access → page fault

CPU accesses virtual address → PTE not present → Page fault

3 Find the VMA

VMA search Maple / RB tree →
- No VMA → SIGSEGV
- VMA found → check flags

4 Check vm_flags

vm_flags r / w / x private / shared →
- Not allowed → SIGSEGV
- Allowed → Week 8

5 How is the page backed?

File-backed (a.out, libc.so)
- VMA has vm_file + vm_pgoff
- Kernel finds the page in page cache or storage

Anonymous (heap, anon mmap)
- No backing file
- Zero-filled page on demand

BSS (zero-fill in executable)
- p_memsz > p_filesz: not in file
- Zero-filled page on demand

Stack
- No backing file; grows on demand (guard page)

Key Takeaway

On a page fault the kernel searches the VMA, then uses it to validate the address, check permission, and find the backing information; the rest of the handling is Week 8.

Checkpoint: Process VAS

Key Points

- **Segments become VMAs**
  The kernel maps **PT_LOAD segments**; sections are for the linker.
- **A VMA = one permission + one backing**
  RELRO or **.bss** can split one segment into several VMAs.
- **Addresses move every run**
  PIE and **ASLR** randomize the layout; the order stays the same.
- **The kernel indexes VMAs**
  Each maps line is one **vm_area_struct**; a fault outside every VMA gives **SIGSEGV**.

Checkpoint: Process VAS

Try to answer each question first, then check the answer.

1 Does the kernel build VMAs from ELF sections or segments?

A. **Segments** (PT_LOAD). Sections are mainly for the linker.

2 Why can one segment become several VMAs?

A. A VMA has **one permission set and one backing**: RELRO or .bss splits it.

3 Why do the addresses in maps change on every run?

A. **PIE** gives a random load bias and **ASLR** randomizes regions; the order stays.

4 What does one line of /proc/<pid>/maps describe?

A. One **vm_area_struct**: address range, permissions, file offset and file.

5 What happens on an access outside every VMA?

A. The fault handler finds no VMA, so the process gets **SIGSEGV**.

6 How does Linux 6.1+ index the VMAs of a process?

A. **mm_struct** holds them in a **Maple Tree** (earlier: red-black tree plus list).

Key Takeaway

Can you explain how an ELF file becomes the VMAs of a running process?

PART 3: Memory Management

- **Week 7**

- Process Virtual Address Space (VAS)
  - Where does a program's memory live?
- **Dynamic VAS Management**
  - How do *brk()* and *mmap()* reshape the address space at runtime?
- Inside the Allocator
  - How does ptmalloc manage memory, and what are the performance-footprint trade-offs?

From Static Layout to Dynamic Growth

Key Points

- **Initial layout after execve**
  The loader maps the code and data; the kernel sets up the stack, and the dynamic linker maps shared libraries. **No [heap] VMA yet**.
- **Every region is a VMA**
  Each mapped region is a **VMA** in the kernel; see them in **/proc/<pid>/maps**.
- **Small requests: heap**
  The **first malloc()** creates the heap with **brk()**; later requests reuse free memory or raise **vm_end**.
- **Large requests: new mapping**
  mmap() creates a new anonymous VMA, an extra line in /proc/<pid>/maps.

How can malloc() reshape the address space?

Initial layout after execve vs. layout while the program runs

Initial (after execve)

- **Stack** (rw-p)
- ...
- **Shared library** libc.so (r-xp)
- ...
- **No [heap] yet** brk is set, size 0; created at the first malloc()
- ...
- **.data / .bss** (rw-p)
- **.text** (r-xp)

```
void *p =
malloc(size);
```

- The **first** malloc() makes the allocator ask the kernel: **brk()** creates the [heap] VMA.
- Later, malloc() first **reuses** free memory in the heap.
- If more is needed: **brk()** may extend the heap, or **mmap()** may add a new mapping.
- So the address space **may or may not change**.

Observe the changes
Compare `/proc/<pid>/maps` before and after: each line is a **VMA**.

During execution

- **Stack** (rw-p)
- ...
- **Shared library** libc.so (r-xp)
- ...
- **New mmap region** new anonymous VMA
- ...
- **Heap** created by brk(); vm_end may grow
- ...
- **.data / .bss** (rw-p)
- **.text** (r-xp)

Key Takeaway

malloc() may reuse allocator memory, create or extend the heap with brk(), or add a new mapping with mmap(); the VMAs in /proc/<pid>/maps show which.

How Does the Kernel Find Address Space?

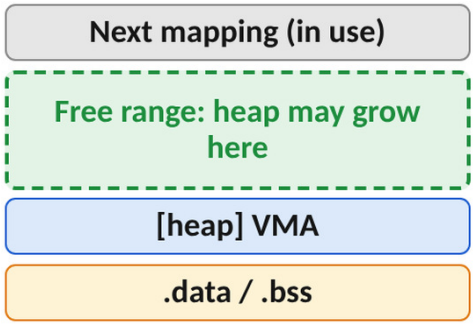
Key Points

- **brk() extends the end**
  It only **moves the end** of the heap. There is **no search**.
- **mmap() finds a free gap**
  The kernel looks for a **free range** that fits, in the mmap region.
- **The VMA tree makes it fast**
  Each node remembers its **largest gap** (Maple Tree, Linux 6.1+; rbtree before).
- **Then it adds a VMA**
  The new VMA is **inserted** or **merged** with a neighbor. **No physical pages** yet.

How does the kernel find address space?

brk() only extends the end; mmap() searches for a free gap

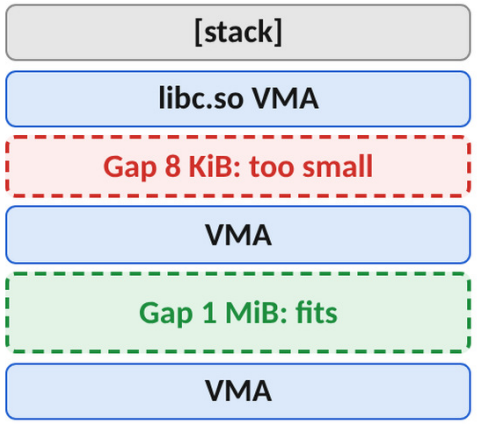
1 brk(): extend the end



If the next range is taken, **brk() fails**.

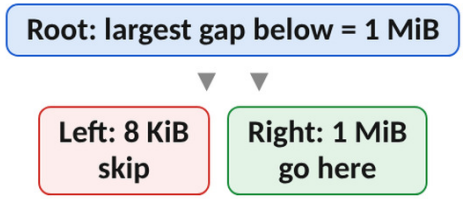
So the heap stays **one contiguous VMA**.

2 mmap(): find a free gap get_unmapped_area



The kernel can use **any free gap**, not only the end.

3 Why is the search fast?



Each tree node remembers the largest gap below it: **Maple Tree** (Linux 6.1+), earlier an **rbtree**. The kernel **skips subtrees** that are too small instead of scanning every VMA.

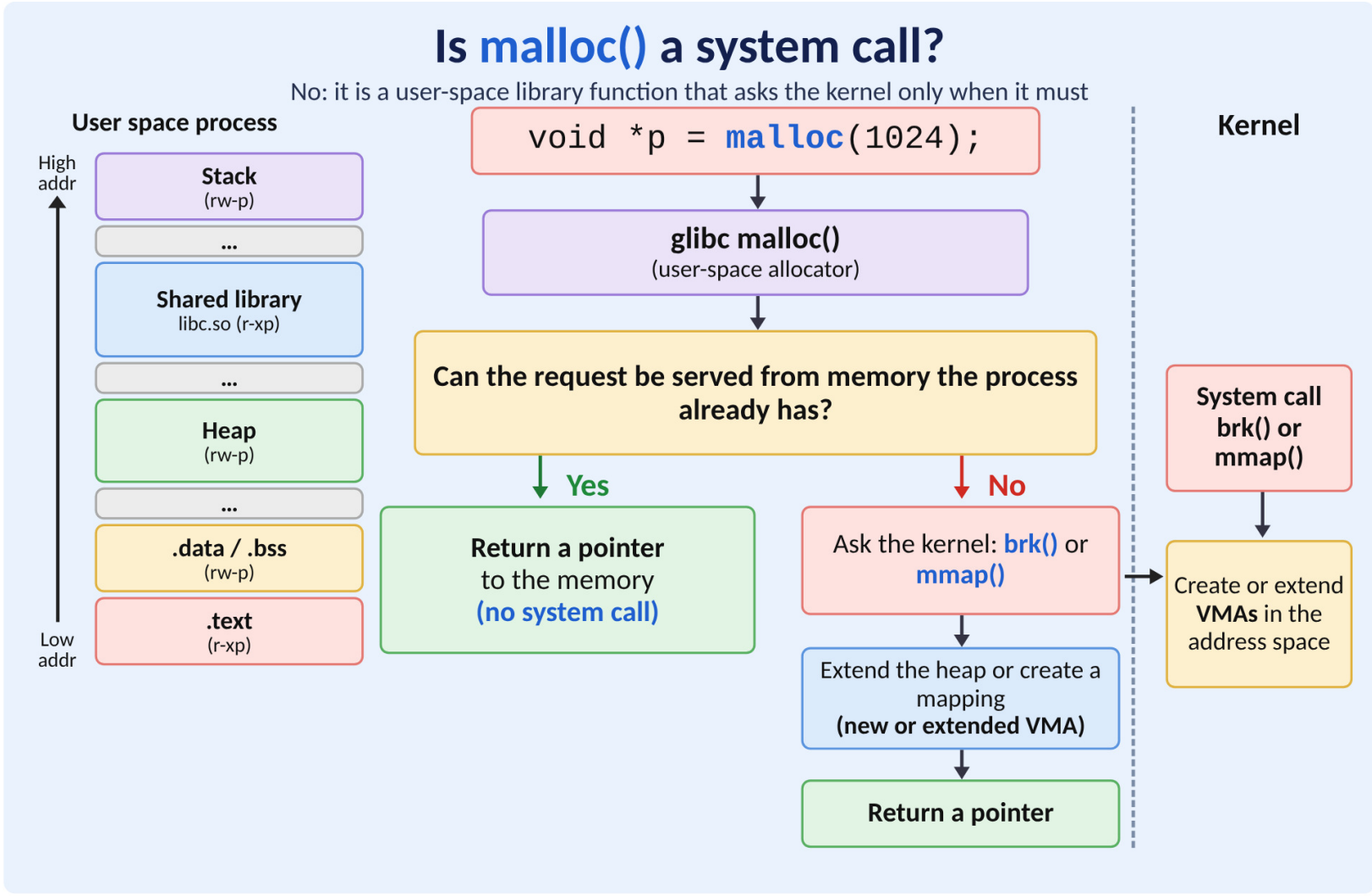
Key Takeaway

brk() only extends the end of the heap, while mmap() lets the kernel search the VMA tree for any free gap that fits; physical pages come later.

Is malloc() a System Call?

Key Points

- **malloc() is a user-space function**
  It lives in the **C library** (e.g., glibc), **not in the kernel**.
- **Works inside acquired memory**
  It reuses **free chunks** or the **top chunk** within the address space the process already has.
- **Kernel only when needed**
  For more memory, malloc() asks the kernel via **brk()** or **mmap()**.
- **Result: usually no system call**
  If acquired memory suffices, a pointer is returned **without entering the kernel**.



Key Takeaway

malloc() is a user-space allocator, not a system call; brk() or mmap() are used only when additional memory is needed.

Observing malloc() with strace

Key Points

- **strace shows system calls only**
  It **cannot see** library functions such as malloc() and free().
- **malloc()/free() are glibc, not syscalls**
  They run in **user space**; only the calls they trigger reach the kernel.
- **What does appear: brk, mmap, munmap**
  brk() grows the heap; the 10 MiB request gets its own **mmap** region; free(p3) **unmaps** it.
- **free(p1), free(p2): no syscall**
  The freed chunks **stay inside the allocator** for reuse.

Why does strace never show malloc()?

strace traces system calls only; malloc() and free() are glibc library functions

1 a.out (C source)

```c
#include <stdio.h> ...
int main(void) {
  char b[32];
  int n = snprintf(b, sizeof b,
        "PID = %d\n", getpid());
  write(1, b, n);

  void *p1 = malloc(1024);
  void *p2 = malloc(1024);
  void *p3 = malloc(10<<20);
  printf("p1=%p p2=%p p3=%p\n",
         p1, p2, p3);
  free(p1); free(p2); free(p3);
}
```

2 strace output (excerpt, PID 1354)

```
$ strace ./a.out
execve("./a.out", ...) = 0
brk(NULL)               = 0x557b93fa8000  # initial break
... (loader / libc setup)
write(1, "PID = 1354\n", 11) = 11
# ---- first malloc() ----
brk(0x557b93fc9000) = 0x557b93fc9000  # heap +0x21000
mmap(NULL, 10489856, ...
  MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)
                    = 0x7f468b9ff000  # p3
write(1, "p1=0x557b93fa82a0 p2=...", 54)
munmap(0x7f468b9ff000, 10489856) = 0
                                  # free(p3)
exit_group(0)
```

# call		syscall seen by strace
p1 = malloc(1024)	->	**brk() heap grows by 0x21000**
p2 = malloc(1024)	->	no syscall (reuses the heap)
p3 = malloc(10<<20)	->	**mmap() its own region**
free(p1), free(p2)	->	no syscall (stay in allocator)
free(p3)	->	**munmap() region returned**

Example output: Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18; addresses vary (ASLR).

Key Takeaway

malloc() and free() are user-space glibc functions, so only the brk(), mmap(), munmap() they trigger appear in strace; small frees never leave the allocator.

Why Doesn’t malloc() Call the Kernel Every Time?

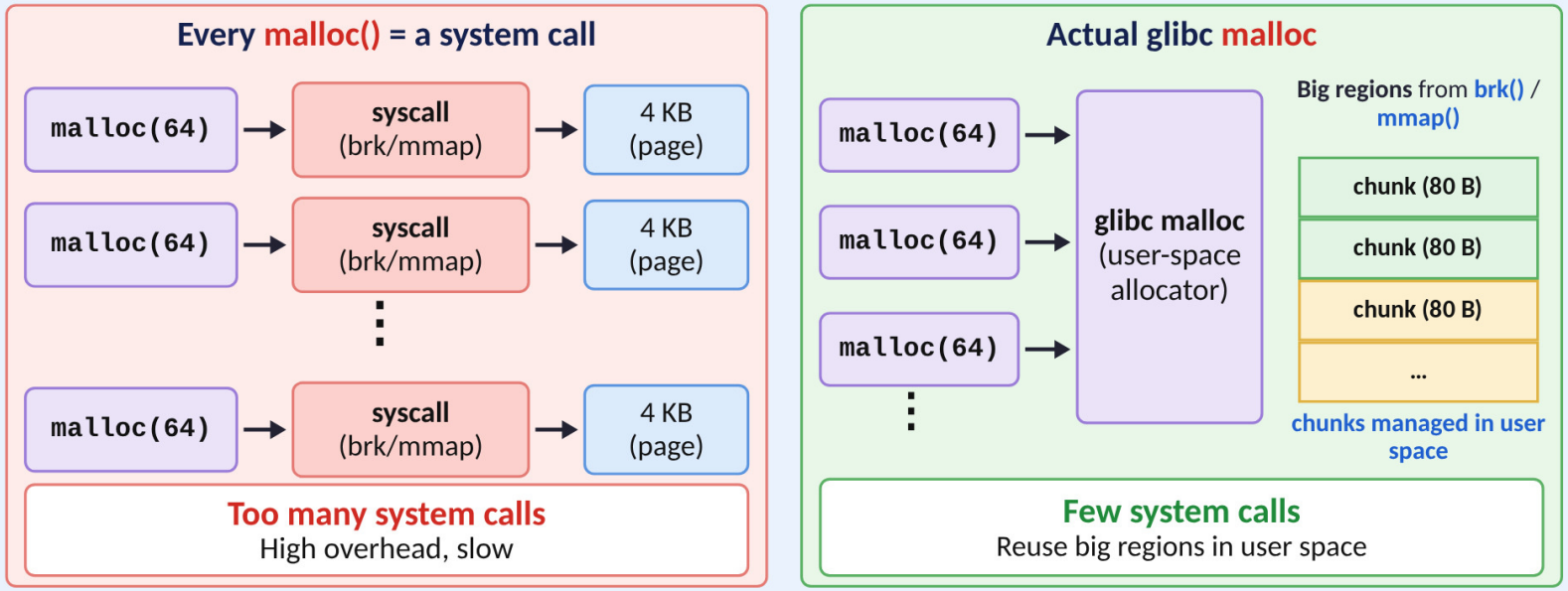
Key Points

- **System calls have overhead**
  Each one switches **user ↔ kernel** mode, and the kernel validates arguments and updates its data structures.
- **Requests are small**
  Tens or hundreds of bytes (e.g., malloc(64)), while memory is managed in **pages** (e.g., 4 KB).
- **A user-space allocator absorbs this**
  It gets **larger regions** from the kernel via brk() or mmap() when needed.
- **Small chunks are carved in user space**
  The region is **split and reused**, so kernel calls stay rare.

Why does malloc() not call the kernel every time?

Small, frequent allocations vs. page-granularity system calls

```
for (int i = 0; i < 10000; i++) {
    p[i] = malloc(64);
}
```



Q. malloc(64) 10,000 times: **brk()/mmap()** 10,000 times?
A. No: far fewer; glibc splits big regions into chunks in user space.

Key Takeaway

A user-space allocator takes large regions from the kernel and serves small requests itself, so malloc(64) 10,000 times causes far fewer than 10,000 system calls.

What is brk()?

Key Points

- **brk() moves the program break**
  The break is the **end of the traditional heap**; raising it extends the heap VMA.
- **No physical pages yet**
  Pages are allocated **on demand**, via page faults.
- **malloc() extends more than asked**
  malloc(1024) does not call brk() for exactly 1 KiB; here the break grows by **0x21000 (~132 KiB)**.
- **The extra space stays in the allocator**
  It serves future allocations, so **free(p) does not lower the break**.

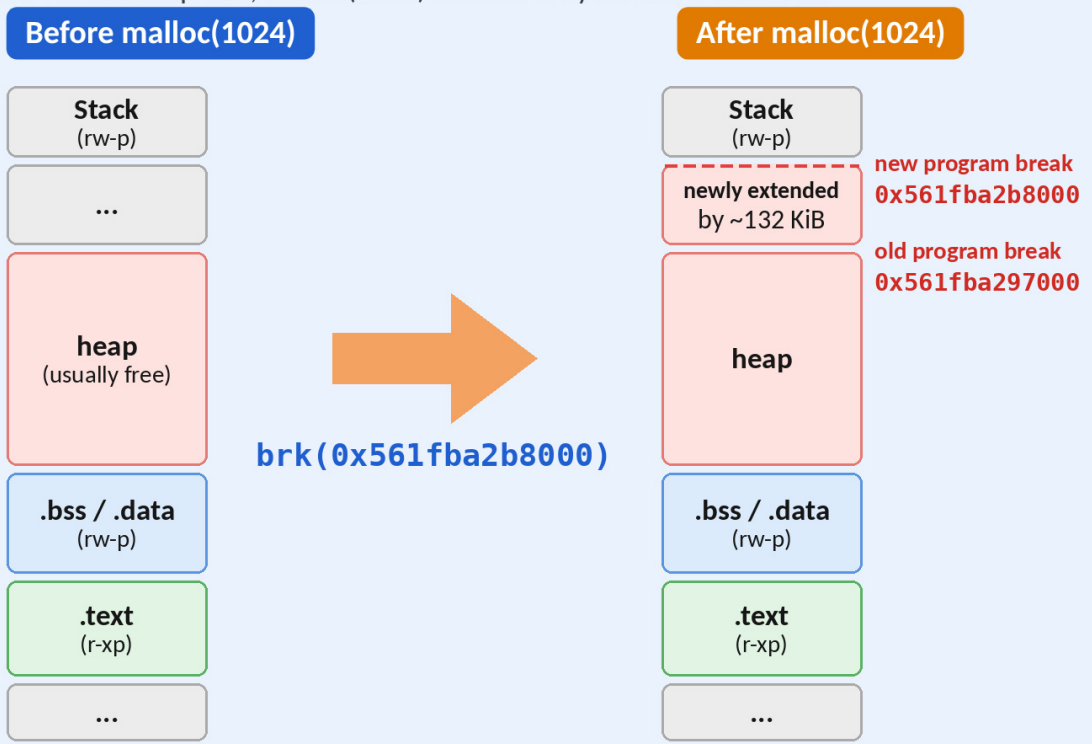
What does brk() do to the heap?

As on the previous slide, brk() only moves the heap end; malloc(1024) extends it by far more than 1 KiB

```c
int main(void) {
  void *p = malloc(1024);
  if (!p) return 1;
  write(1, "allocated\n",
        10);
  free(p);
  return 0;
}
```

```
$ strace -e brk ./a.out
brk(NULL) = 0x561fba297000
brk(0x561fba2b8000)
    = 0x561fba2b8000
# free(p): no brk call
```

Example output: Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18; addresses vary (ASLR).



Old break	0x561fba2b8000 − 0x561fba297000	New break
0x561fba297000	= 0x21000 = 135,168 B (≈ 132 KiB)	0x561fba2b8000

Key Takeaway

brk() changes the program break so the heap can grow; malloc() takes a much larger region than requested and manages the rest in user space to avoid future system calls.

Observing brk() in /proc/<pid>/maps

Key Points

- **At execve: break set, no heap yet**
  The break is initialized, but the **[heap] VMA may not exist** yet (heap size = 0).
- **1st malloc() creates the heap**
  malloc(1024) calls **brk()**, which moves the break up and **creates the [heap] VMA**.
- **2nd malloc() reuses it**
  It uses existing heap space; the **VMA and break usually do not change**.
- **Large requests may skip brk()**
  Some allocations (e.g., large ones) use **mmap()** instead.

How does the heap appear step by step?

Following a.out with /proc/<pid>/maps and strace

```
wait_key("before");
void *p1 = malloc(1024);   wait_key("after 1st");
void *p2 = malloc(1024);   wait_key("after 2nd");
```

1 After exec

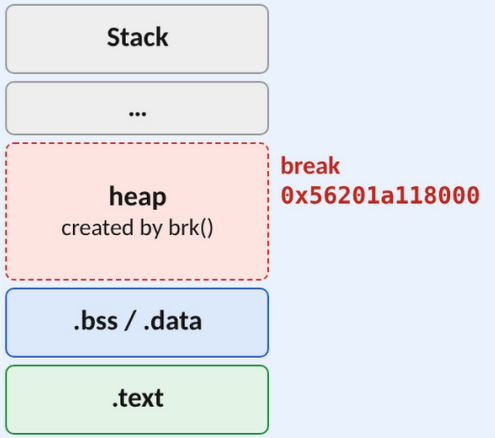
Break initialized, but the **[heap] VMA may not exist** yet.

```
$ grep heap /proc/<pid>/maps
(no [heap] line)
brk(NULL) = 0x56201a0f7000
```

2 After 1st malloc

malloc() calls brk() and **creates the [heap] VMA.**

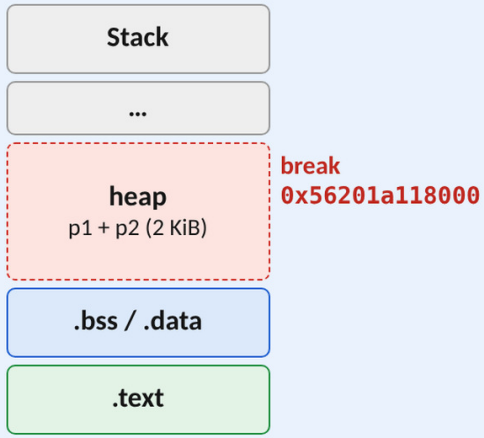
```
56201a0f7000-56201a118000
rw-p ... [heap]
brk(0x56201a118000)
```



3 After 2nd malloc

Reuses heap space: **VMA and break usually stay the same.**

```
56201a0f7000-56201a118000
rw-p ... [heap]
(no brk call)
```



Example output: Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18; addresses vary (ASLR).

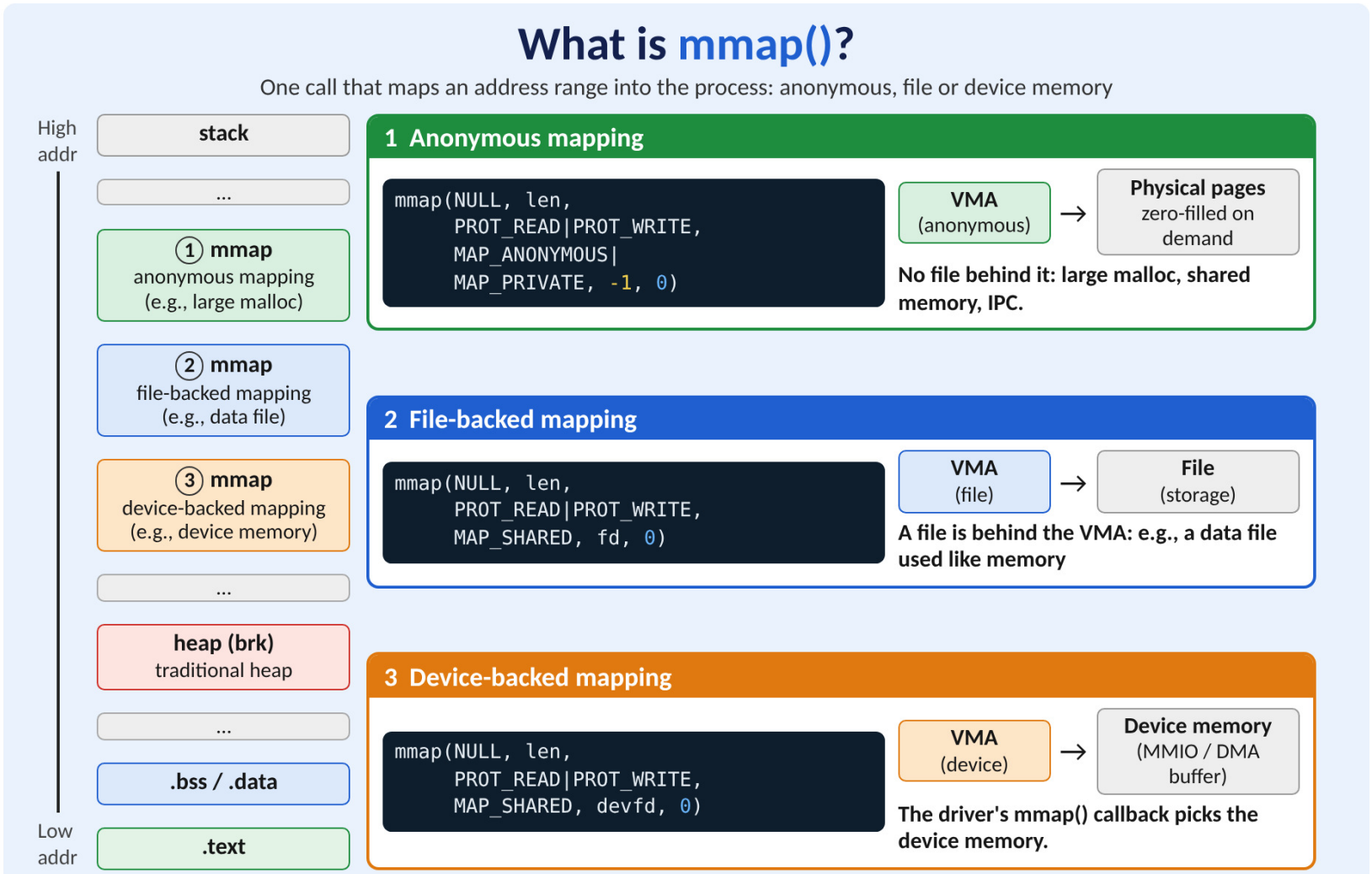
Key Takeaway

The first malloc() creates the [heap] VMA via brk() and takes more than requested; later small mallocs reuse that space without changing the VMA.

What is mmap()?

Key Points

- **mmap() maps an address range**
  As on the previous slides, it finds a free gap and adds a **VMA** (or merges with a neighbor); here: the **kinds of backing**.
- **Three kinds of backing**
  Anonymous memory, a **file**, or **device** memory.
- **Page-based, on demand**
  Physical pages are allocated **on demand** (e.g., via page faults).
- **Plain loads and stores**
  The process uses the VMA with **ordinary load/store** instructions.



Key Takeaway

mmap() maps a virtual address range, typically creating a new VMA; the backing can be anonymous, file or device, with pages allocated on demand, beyond the traditional heap (brk).

Anonymous Mapping Example

Key Points

- **Memory with no file behind it**
  An **anonymous mapping** is not backed by a file; its pages start **zero-filled**.
- **Call mmap() with MAP_ANONYMOUS**
  With **fd = -1**, a successful call **creates a 1 MiB VMA**; no physical pages yet.
- **First access → page fault**
  The kernel allocates **zero-filled pages** on demand.
- **munmap() releases it**
  The mapping is **removed** and its pages can be reclaimed.
- **Why use it?**
  Large blocks get **their own VMA**, and **munmap()** returns them to the kernel at once.

How does an anonymous mmap() work?

Map 1 MiB, touch it, release it

1 Example: anonymous mmap()

```c
size_t len = 1 << 20;  // 1 MiB

// 1. create anonymous mapping
void *p = mmap(NULL, len,
    PROT_READ | PROT_WRITE,
    MAP_PRIVATE | MAP_ANONYMOUS,
    -1, 0);
printf("mmap() returned: %p\n", p);

// 2. use the memory
memset(p, 0xAA, len);
printf("first write done\n");

// 3. release the mapping
munmap(p, len);
printf("munmap() done\n");
```

```
$ ./anon_mmap
mmap() returned: 0x7fa4a3d00000
first write done
munmap() done
```

1 Call mmap()
Ask for 1 MiB. On success the kernel adds a 1 MiB VMA. **No physical pages yet.**
VMA: created, empty

2 First write: fault
memset() faults per page. **Kernel adds zero-filled pages.**
VMA: pages allocated

3 Use the memory
Normal load/store. **No more faults on resident pages.**
pages resident

4 munmap()
Removes the mapping. **The pages can be reclaimed.**
mapping removed

Key parameters

Parameter	Meaning
`addr`	NULL (kernel chooses)
`length`	1 << 20 (1 MiB)
`prot`	PROT_READ | PROT_WRITE
`flags`	MAP_PRIVATE | MAP_ANONYMOUS
`fd, offset`	-1, 0 (no file)

Example output: Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18; addresses vary (ASLR).

Key Takeaway

An anonymous mmap() creates a VMA first and allocates zero-filled pages on demand via page faults; munmap() removes the mapping so its pages can be reclaimed.

File-Backed Mapping Example

Key Points

- **Open and map the file**
  open() gives an **fd**; mmap() with it creates a **file-backed VMA**.
- **First access → page fault**
  The kernel maps the page from the **page cache**, reading it from storage if missing.
- **Stores dirty the page**
  With **MAP_SHARED**, a store makes the cached page **dirty**; it is **written back later**.
- **Sync and release**
  msync(MS_SYNC) waits for the write-back; **munmap()** and **close()** release the mapping and fd.
- **Why use it?**
  It avoids the **copy** made by read()/write(), but is **not always faster**; it depends on the workload.

How does a file-backed mmap() work?

Map data.bin, modify it through memory, sync it back

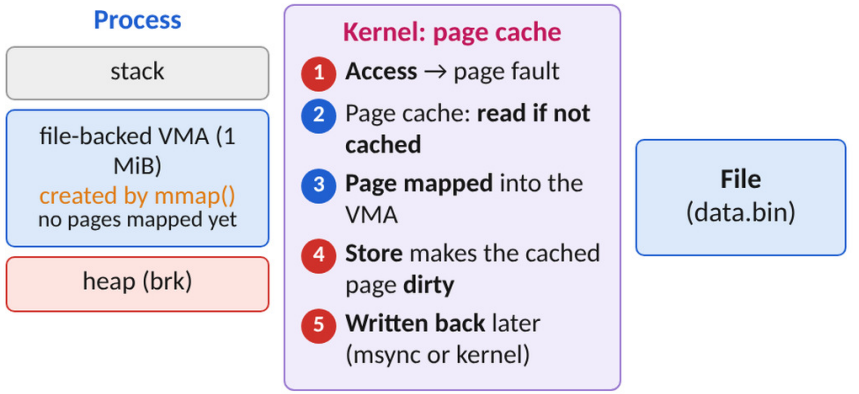
1 Example: file-backed mmap()

```
int fd = open("data.bin",
    O_RDWR | O_CREAT, 0644);
ftruncate(fd, 1 << 20);  // 1 MiB

// map the file
void *p = mmap(NULL, 1 << 20,
    PROT_READ | PROT_WRITE,
    MAP_SHARED, fd, 0);

strcpy(p, "Hello, mmap file!\n");
msync(p, 1 << 20, MS_SYNC);
munmap(p, 1 << 20);
close(fd);
```

```
$ ./file_mmap
$ head -c 17 data.bin
Hello, mmap file!
```

Example output: Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18; addresses vary (ASLR).

Data flow



Key parameters

Parameter	Meaning
`addr`	NULL (kernel chooses)
`length`	1 << 20 (1 MiB)
`prot`	PROT_READ | PROT_WRITE
`flags`	MAP_SHARED (writes reach the file)
`fd`	open file descriptor
`offset`	0 (page-aligned)

Key Takeaway

mmap() with a file descriptor creates a file-backed mapping accessed with normal memory instructions through the page cache; with MAP_SHARED, changes are written back to the file.

Device-Backed Mapping Example

Key Points

- **Memory that belongs to a device**
  Points at **device memory** (MMIO registers or a DMA buffer), not a file.
- **Open, then mmap()**
  open() gives an fd; mmap() calls the **driver's mmap callback** to set up the **VMA**.
- **Access like memory**
  Usually mapped at **mmap() time**: **no page fault** (some drivers fault on demand) and **no page cache**.
- **Release**
  munmap() and **close()** release the mapping and the fd.
- **Why use it?**
  Reach the device **without a system call per access**; the driver decides what is exposed.

How does a device-backed mmap() work?

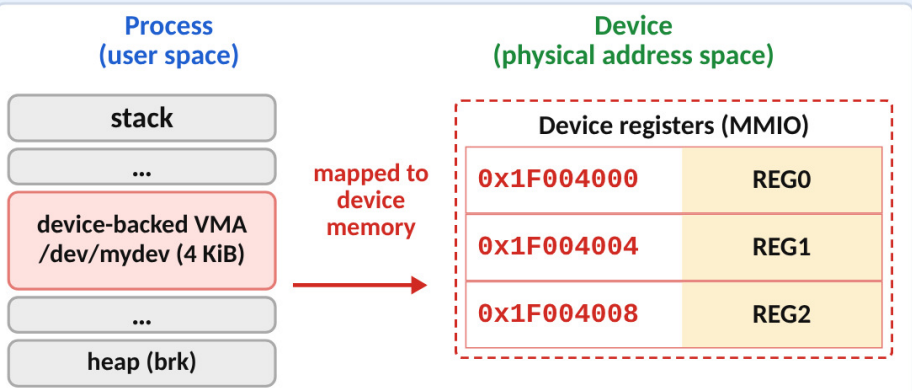
The driver, not the kernel core, decides what memory the mapping refers to

```c
int fd = open("/dev/mydev", O_RDWR);
void *p = mmap(NULL, 4096,
    PROT_READ | PROT_WRITE,
    MAP_SHARED, fd, 0);

volatile uint32_t *regs = p;
uint32_t val0 = regs[0];  // REG0
printf("REG0 = 0x%08x\n", val0);
regs[1] = 0x1;            // REG1
printf("REG1 = 0x%08x\n", regs[1]);

munmap(p, 4096);
close(fd);
```

```
$ ./dev_mmap
REG0 = 0x00000005
REG1 = 0x00000001
```

⚠ **Conceptual example:** needs a driver that supports mmap(); /dev/mydev is a dummy device.

Memory mapping (MMIO example)



Key parameters

Parameter	Meaning
`addr`	NULL (kernel chooses)
`length`	4096 (4 KiB here; driver-limited, multiple of page size)
`prot`	PROT_READ | PROT_WRITE
`flags`	MAP_SHARED (typical for devices)
`fd`	open("/dev/mydev", O_RDWR)
`offset`	0 (page-aligned, device-specific)

Register addresses are illustrative, not real hardware.

Key Takeaway

Device-backed mmap() maps device memory (MMIO registers or a DMA buffer) into the process, set up by the driver's mmap() callback.

Device-Backed Mapping Without Device Driver Example

Key Points

- **Physical memory seen as a file**
  /dev/mem exposes physical addresses; the mmap() **offset** is the physical address (page-aligned).
- **Open it, then mmap()**
  open() with **O_RDWR** (typically O_SYNC); mmap() maps that page with **remap_pfn_range()**.
- **Access like memory**
  The page table already points at the page, so there is **no page fault** and **nothing is allocated**.
- **Release**
  munmap() and **close()** release the mapping and the fd.
- **Why use it?**
  Quick **hardware bring-up and debugging** only; it needs **root** and the kernel may block it.

How does /dev/mem map a physical address?

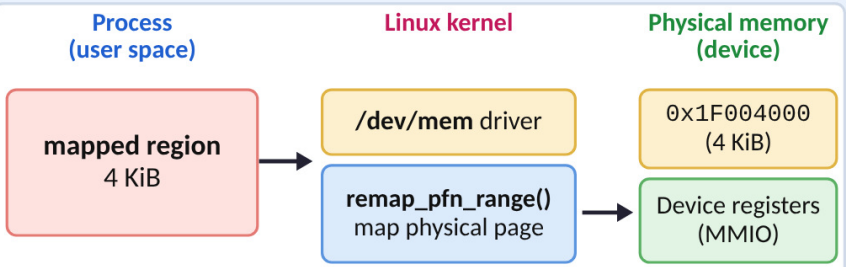
Mapping existing physical pages into user space (conceptual)

⚠ **Conceptual example: do not run without verified hardware addresses.** Needs root; kernel config may block it.

```c
off_t phys = 0x1F004000;  // target PA
int fd = open("/dev/mem",
              O_RDWR | O_SYNC);

void *p = mmap(NULL, 4096,
    PROT_READ | PROT_WRITE,
    MAP_SHARED, fd, phys);

volatile uint32_t *regs = p;
uint32_t val0 = regs[0];  // REG0
printf("REG0 = 0x%08x\n", val0);
regs[1] = 0x1;            // REG1
printf("REG1 = 0x%08x\n", regs[1]);

munmap(p, 4096);
close(fd);
```

```
$ sudo ./dev_mem_demo
REG0 = 0x00000005
REG1 = 0x00000001
# illustrative, device-dependent
```

Mapping flow



Key parameters

Parameter	Meaning
`addr`	NULL (kernel chooses)
`length`	4096 (4 KiB here; rounded up to the page size)
`prot / flags`	RW / MAP_SHARED
`fd`	open("/dev/mem", O_RDWR | O_SYNC)
`offset`	0x1F004000 (page-aligned PA)

Not for normal applications. Debug/bring-up only; may be blocked (CONFIG_STRICT_DEVMEM).

Key Takeaway

/dev/mem maps a physical address range into the process with remap_pfn_range() and MAP_SHARED; PTEs are set at mmap() time, so there is no page fault on first access.

Observing mmap() with /proc/<pid>/maps

Key Points

- **File-backed mapping**
  Has a **pathname and inode**, dev # ≠ 00:00, typically **rw-s** (shared).
- **Anonymous mapping**
  No pathname, inode 0, dev 00:00, typically **rw-p** (private).
- **Device mapping**
  /dev/zero (MAP_SHARED) is backed by an internal shmem file, so it shows **(deleted)**; real devices show their **device path**.
- **No malloc, no [heap]**
  This program never calls malloc(), so the map has **no [heap] VMA**.

What do the three mapping types look like in maps?

Each mapping type leaves a different fingerprint in the pathname, inode and dev columns.

1 mmap_demo.c

```c
// 1. file-backed mapping
fd = open("test.dat",
          O_RDWR|O_CREAT, 0644);
ftruncate(fd, 4096);
mmap(NULL, 4096,
     PROT_READ|PROT_WRITE,
     MAP_SHARED, fd, 0);

// 2. anonymous mapping
mmap(NULL, 4096,
     PROT_READ|PROT_WRITE,
     MAP_PRIVATE|MAP_ANONYMOUS,
     -1, 0);

// 3. device mapping
dfd = open("/dev/zero", O_RDWR);
mmap(NULL, 4096,
     PROT_READ|PROT_WRITE,
     MAP_SHARED, dfd, 0);

sleep(60);  // keep alive
```

Example output: Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18; addresses vary (ASLR).

2 /proc/<pid>/maps (real run, excerpt)

```
$ ./mmap_demo &
$ cat /proc/<pid>/maps   (excerpt)
55bc1014d000-55bc1014e000 r-xp 00001000 fe:00 8404995 /t/mmap_demo
55bc10150000-55bc10151000 rw-p 00003000 fe:00 8404995 /t/mmap_demo
(no [heap]: this program never calls malloc)
7f77d97bc000-7f77d97bd000 rw-s 00000000 00:01 1025 /dev/zero (deleted)
7f77d97bd000-7f77d97be000 rw-p 00000000 00:00 0
7f77d97be000-7f77d97bf000 rw-s 00000000 fe:00 8404997 /t/test.dat
```

3 Columns in /proc/<pid>/maps

Column	Meaning
address	start–end of the VMA
perms	r/w/x; p = private (not shared), s = shared
offset	byte offset in the file (0 if anonymous)
dev	fs major:minor (00:00 = anonymous)
inode	file inode (0 if anonymous)
pathname	mapped file or device (empty if anonymous)

Key Takeaway

Read the perms, dev, inode and pathname columns to tell a file-backed, an anonymous and a device-backed mapping apart.

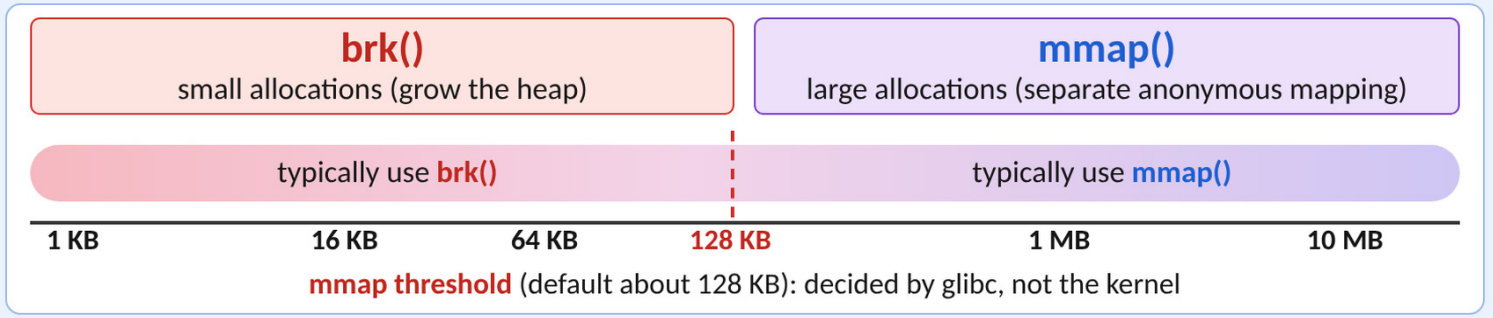
malloc() Uses brk() and mmap(): mmap Threshold

Key Points

- **Small → brk(), large → mmap()**
  Below the threshold glibc **grows the heap**; above it, it creates a **separate anonymous mapping**.
- **A glibc policy, not a kernel rule**
  The **user-space allocator** compares the request size with the **mmap threshold**.
- **Default about 128 KB, but dynamic**
  It can vary with past allocation and free behavior; **mallopt()** sets it and disables the adaptation.
- **Different free() behavior**
  Heap chunks are kept for reuse; **mmap** regions are returned with **munmap()**.

When does malloc() use brk() and when mmap()?

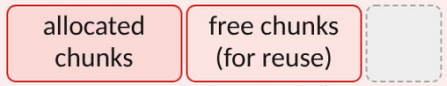
The choice is made by glibc from the request size and the mmap threshold.



brk(): small allocations

Extends the existing heap VMA; no new VMA per allocation. Freed chunks are **kept by the allocator for reuse**.

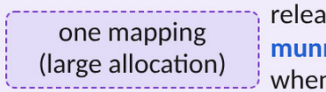
Heap (brk)



heap grows upward with brk() →

mmap(): large allocations

Creates a separate anonymous mapping (new VMA). Freed memory is released with **munmap()** back to the kernel.



released with **munmap()** when freed

mmap threshold (glibc)

Large requests use mmap **only if the heap's free space cannot serve them**. Default: about **128 KB**, but it **adapts dynamically**.

`mallopt(M_MMAP_THRESHOLD, bytes)`
Default 128 KB; adapts up to **32 MiB**. Setting it **disables the adaptation**.

Main arena grows the heap with brk(); **thread arenas** use heaps created by mmap(), so an mmap-area region can still be ptmalloc-managed.

Key Takeaway

glibc picks brk() for smaller and mmap() for larger requests based on size, threshold and allocator state; the threshold is an allocator policy tunable with mallopt().

Observing mmap Threshold is Dynamic

Key Points

- **Below the threshold: heap**
  malloc(100 KB) is served by **brk()** (heap +0x21000).
- **Above the threshold: mmap**
  The first malloc(200 KB) is served by **mmap()** (208,896 B).
- **free() raises the threshold**
  Freeing that mmap chunk **sets the threshold to its size**, so the next 200 KB comes from the heap.
- **Fixing it turns this off**
  Setting the tunable or **mallopt() disables dynamic adjustment**; 128 KB is only the starting value.

Is the mmap threshold really fixed at 128 KB?

Allocate and free a 200 KB block, then watch which system call serves the next 200 KB request.

1 mmap_threshold.c (excerpt)

```
a = malloc(100*KB);    // 1: heap
b = malloc(200*KB);    // 2: > 128 KB
free(b);               // 3: munmap
c = malloc(200*KB);    // 4: again
free(c); free(a);      // 5: free all
```

2 strace (real run, default glibc)

```
brk(NULL) = 0x55c36a714000          # 1
brk(0x55c36a735000) = 0x55c36a735000  # +0x21000
mmap(NULL, 208896, ...) = 0x7f895c7d8000 # 2
munmap(0x7f895c7d8000, 208896)          # 3
brk(0x55c36a780000) = 0x55c36a780000  # 4: heap grows
brk(0x55c36a735000) = 0x55c36a735000  # 5: trim
```

3 What the allocator decided

#	Request	Served by	Threshold
1	malloc 100 KB	heap (brk)	128 KiB
2	malloc 200 KB	mmap	128 KiB
3	free that block	munmap	→ 208,896 B
4	malloc 200 KB again	heap (brk)	208,896 B
5	free all	trim heap	trim = 2×

free() of an mmap'd chunk **raises the threshold** to its size (208,896 B), up to 32 MiB. Row 4: heap had no room, so **brk()** grew it, not mmap().

4 Threshold fixed at 1 MiB

```
$ GLIBC_TUNABLES=\
    glibc.malloc.mmap_threshold=1048576 ./mt
write(1, "2: malloc 200 KB\n", 17)
brk(0x55841ce2a000) = 0x55841ce2a000  # heap
# no mmap / munmap
```

Setting the tunable (or mallopt) turns dynamic adjustment **off**; "≥ 128 KB → mmap" is only the starting value.

glibc 2.39, x86-64, Linux 6.18; addresses vary (ASLR).

Key Takeaway

The 128 KB mmap threshold is only an initial value: glibc raises it after freeing an mmap chunk, and fixing it with a tunable or mallopt() disables that adaptation.

Virtual Size vs Resident Size

Key Points

- **malloc() reserves address space**
  A new **100 MiB anonymous VMA** is created (via mmap), so **VmSize** increases right away.
- **RSS stays small**
  Physical pages are **not allocated until they are touched**.
- **Touching pages makes them resident**
  Writing every page triggers **page faults**; RSS rises by 100 MiB (+102,400 kB).
- **Read both numbers**
  VmSize shows what was reserved, **VmRSS** what is really in memory.

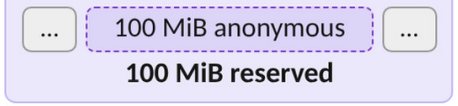
Does malloc(100 MB) use 100 MB of RAM?

Virtual size (VmSize) and resident size (RSS) change at different moments.

malloc(100 MiB)

```
p = malloc(100 MiB);
```

Virtual mapping (VMA)



100 MiB reserved

Resident memory (RSS)

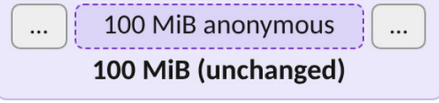
Few pages resident

Virtual size grows, but RSS **stays almost unchanged**.

Write every page

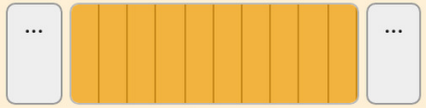
```
memset(p, 0xAA, 100 MiB);
```

Virtual mapping (VMA)



100 MiB (unchanged)

Resident memory (RSS)



100 MiB resident

RSS **grows by 100 MiB**: pages come in on demand (page faults).

/proc/<pid>/status

After malloc (before touching)

```
$ cat /proc/<pid>/status
VmSize: 105116 kB # 100 MiB
VmRSS:  1352 kB # little change
```

After touching all pages

```
$ cat /proc/<pid>/status
VmSize: 105116 kB # same
VmRSS:  103752 kB # +102,400 kB
```

103,752 − 1,352 = **102,400 kB** = 100 MiB: VmRSS grew by the size of the allocation.

Example output: Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18; numbers vary by system.

Key Takeaway

Allocating memory grows the virtual size immediately, but resident size grows only when pages are touched (on-demand paging).

Observing Memory Usage with smaps or smaps_rollup

Key Points

- **RSS counts shared pages in full**
  Pages such as **libc** are mapped by many processes, and each process's **Rss** includes all of them.
- **Shared vs Private**
  Shared pages are also mapped by another process; **Private** pages only by this one. **Rss = Shared + Private**.
- **Pss: a fair share**
  Pss divides each shared page among its sharers, so **summing Pss** matches real usage.
- **smaps shows them**
  smaps reports per mapping, **smaps_rollup** for the whole process.

How do smaps and smaps_rollup report resident memory?

Rss counts shared pages in full, so we read Shared, Private and Pss.

1 smaps (one mapping)

```
$ cat /proc/<pid>/smaps
200000000000-200000010000 rw-p
Size:                 64 kB
Rss:                  16 kB
Pss:                   8 kB
Shared_Dirty:         16 kB
Private_Clean:         0 kB
Private_Dirty:         0 kB
Anonymous:            16 kB
```

2 smaps_rollup (process)

```
$ cat smaps_rollup
Rss:                1816 kB
Pss:                 188 kB
Shared_Clean:       1664 kB
Shared_Dirty:        120 kB
Private_Clean:         0 kB
Private_Dirty:        32 kB
Anonymous:           136 kB
```

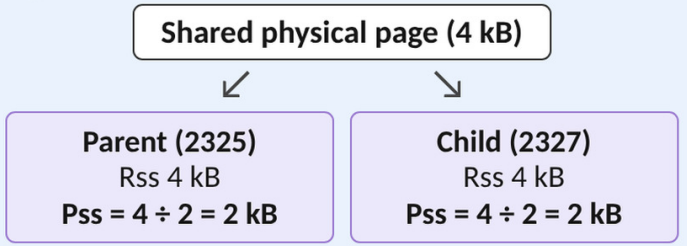
Check: Rss 1816 = 1664 + 120 + 0 + 32 (Shared + Private).
Pss 188 ≥ Private 32.
Shared_Dirty: pages shared with the child after fork, until one of them writes.

Example output: Ubuntu 24.04, glibc 2.39, x86-64, Linux 6.18; numbers vary by system.

3 Field meanings

Field	Meaning
Size	virtual size of this mapping
Rss	resident = Shared_* + Private_*
Pss	Rss with shared pages divided among sharers
Private_*	mapped only by this process
Shared_*	also mapped by another process
Anonymous	resident, not backed by a file

4 Pss: one shared page, two processes



Sum of Rss = 8 kB (double-counted)
Sum of Pss = 4 kB (correct)

Key Takeaway

smaps shows memory per mapping and smaps_rollup per process; use Pss, not Rss, to compare or add up memory of processes that share pages.

Checkpoint: Dynamic VAS

Key Points

- **malloc() is a library call**
  It calls **brk()** or **mmap()** only when it needs more address space.
- **mmap() reserves, a fault fills**
  A successful call creates a **VMA**; physical pages come on **first access**.
- **Small vs large**
  Below the **mmap threshold** the heap grows; above it, **mmap()** gets its own VMA.
- **VmSize vs RSS**
  VmSize counts reserved space, **RSS** resident pages; use **Pss** across processes.

Checkpoint: Dynamic VAS

Try to answer each question first, then check the answer.

1 Is malloc() a system call?

A. **No.** It is a glibc function that calls brk() or mmap() only when it needs more address space.

2 What do brk() and mmap() each change?

A. brk() moves the **end of the heap**; mmap() creates a **new VMA**.

3 Does a successful mmap() allocate physical pages?

A. **No.** It creates the VMA; pages arrive on the **first access** (page fault).

4 When does malloc() use mmap() instead of the heap?

A. For large requests (default **≥ 128 KB**, adjustable) that the heap cannot serve.

5 malloc(100 MiB), never touched: what grows?

A. **VmSize** grows by 100 MiB; **RSS** barely moves.

6 Why add up Pss, not Rss, across processes?

A. Rss counts shared pages **in full** for each process; Pss **splits** them among sharers.

Key Takeaway

Can you explain how malloc() reshapes the address space with brk() and mmap(), and what VmSize and RSS measure?

PART 3: Memory Management

- **Week 7**

- Process Virtual Address Space (VAS)
  - Where does a program's memory live?
- Dynamic VAS Management
  - How do *brk()* and *mmap()* reshape the address space at runtime?
- **Inside the Allocator**
  - How does ptmalloc manage memory, and what are the performance-footprint trade-offs?

Memory Allocators in General

Key Points

- **malloc() is an API**
  The allocator behind it is a **separate implementation** sitting between the application and the kernel.
- **Allocators make different trade-offs**
  Latency, scalability, footprint and **fragmentation** are weighed differently per environment.
- **Case study: glibc ptmalloc**
  Chunks, tcache, bins, arenas and the **top chunk** are its main components.
- **Matters most on embedded devices**
  Limited memory and long-running services make allocator behavior visible in **RSS** and stability.

What is a memory allocator, and why does it matter?

malloc() is only an API; the implementation underneath can be very different.

What is an allocator?

Application
C/C++ program

↕

malloc() / free()
standard API

↕

Allocator
implementation

↕

OS kernel
brk(), mmap()

Why different allocators?

1. **Performance**
   latency, scalability
2. **Memory efficiency**
   fragmentation, RSS
3. **Workload**
   sizes, lifetimes
4. **Environment**
   limits, real-time, security
5. **Design trade-offs**
   speed vs. simplicity

Examples

Allocator	Focus
glibc malloc	general-purpose
jemalloc	low fragmentation
TCMalloc	thread-local cache
mimalloc	fast, good locality
Custom	fixed pools

glibc malloc: ptmalloc2

Derived from dlmalloc (Doug Lea); the **default allocator in GNU/Linux**, built for multi-threaded programs.

Chunks allocation unit	**tcache** per-thread cache	**Bins** free lists by size
Arenas multi-thread	**Top chunk** free end of heap	**brk() / mmap()** obtain memory

Why it matters on embedded devices

- **Limited memory budget** on embedded devices.
- Many **long-running services** (daemons, media, IPC).
- Visible in **RSS, fragmentation, latency, OOM**.
- glibc malloc is the default: **tune and debug** it.

Key Takeaway

malloc() is only an API and allocators differ, so we study glibc ptmalloc: its behavior affects memory use and stability on resource-constrained systems.

Allocation Unit: What is a Chunk?

Key Points

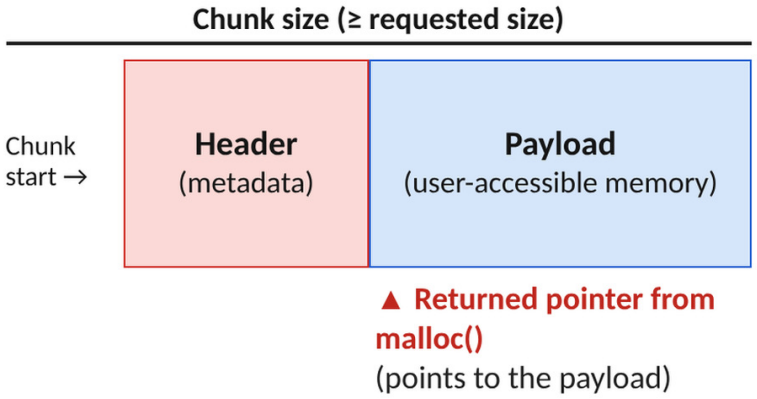
- **Chunk = management unit**
  malloc(size) typically allocates a **chunk larger** than the requested size.
- **Header + payload**
  The **header** holds allocator metadata; the **payload** is the memory the application uses.
- **malloc() returns the payload**
  The returned pointer points to the start of the **payload**, not the header.
- **Size and alignment vary**
  Requests are rounded up to the allocator's **alignment** and **minimum chunk size** (ptmalloc x86-64: **16 B / 32 B**).

What is a chunk?

The allocator manages memory in chunks, not in the exact sizes requested.

Chunk structure

A chunk = **header** (metadata: prev_size, size + flags) + **payload** (user-accessible memory).
malloc() returns a pointer to the **payload**, not to the header.



Chunk size and alignment

Minimum chunk size and alignment **depend on the allocator and the architecture**. The requested size is rounded up to meet them, and different allocators use different chunk formats.

Architecture	Typical alignment	Minimum chunk size
x86 (32-bit)	8 bytes	16 bytes
x86-64 (64-bit)	**16 bytes**	**32 bytes**

Example values for glibc ptmalloc; other allocators may differ.

Alignment: chunk sizes are multiples of it (x86-64: 16 B).

Key Takeaway

A chunk is the allocator's management unit, while the payload is the memory region exposed to the application.

Fragmentation in Memory Allocators

Key Points

- **Internal fragmentation**
  The chunk is larger than the request; the **extra space inside it** is wasted.
- **Overhead = metadata + padding**
  malloc(33) uses a **48 B** chunk (8 B header, 7 B alignment); **malloc(100)** uses **112 B**.
- **External fragmentation**
  Free memory is split into **small non-contiguous blocks** (64 + 48 + 80 = 192 B).
- **A large request may not fit**
  No single free block of 128 B exists, so the allocator may ask the OS for more.

Why does fragmentation matter?

Programs request and free memory of many sizes; the allocator tries to minimize fragmentation and overhead.

Internal fragmentation

The chunk is **larger than the request**; the extra space inside it cannot serve other allocations.

malloc(33) → 48 B chunk

33 B data + 7 B unused + 8 B metadata

malloc(100) → 112 B chunk

100 B data + 4 B unused + 8 B metadata

Overhead = metadata + unused (alignment) space.
Chunk sizes are multiples of 16 B; bars not to scale.

External fragmentation

Free memory exists but is split into **small, non-contiguous blocks**.

Request 128 B ✕ **No contiguous free block of 128 B**, though free memory totals 64 + 48 + 80 = 192 B.

The allocator may then ask the OS for more memory (brk or mmap).

Key Takeaway

Fragmentation wastes memory and can prevent large allocations, even when the total free memory looks sufficient.

Free Chunk Management: Free List

Key Points

- **Allocators keep free lists**
  malloc() must **quickly find** a free chunk of suitable size.
- **Only free chunks are linked**
  The **links live inside** the free chunk's payload; allocated chunks are not in the list.
- **Many organizations exist**
  Singly or doubly **linked lists**, or separate size-based lists (**bins**).
- **Coalescing and splitting**
  Merge adjacent free chunks, or **split** a larger one to fit a request.
- **Design trade-offs**
  Speed, **fragmentation** and **management overhead** pull in different directions.

What is a free list, and what does its design cost?

The allocator keeps only free chunks in lists so it can find one quickly.

1. Singly linked list

- Each chunk points to the **next**
- Simple, low overhead

chunk → chunk → chunk ...

2. Doubly linked list

- Points to **previous and next**
- Fast insertion and removal

chunk ↔ chunk ↔ chunk

3. Size-based lists

One list per size class (bin)

small ≤ 64 B chunk → chunk ...

medium ≤ 1 KB chunk → chunk ...

large > 1 KB chunk → chunk ...

Coalescing and splitting

Coalescing: merge neighbours

free + free ⇒ larger free

Splitting: carve a larger chunk

larger free ⇒ alloc + free

The links are stored inside each free chunk (in its payload).

Design trade-offs

Allocation speed	Faster search (size-based lists) costs overhead
Fragmentation	List policy, coalescing and splitting change it
Management overhead	More metadata and more lists add complexity
Coalesce / split policy	Aggressive vs. conservative: speed vs. fragmentation

Key Takeaway

A free list keeps only free chunks, and its design trades off allocation speed, fragmentation, and management overhead.

Two Performance Metrics: Latency vs Memory Utilization

Key Points

- **Allocation latency**
  Time to serve one malloc/free; watch **mean, tail (p95/p99)** and **throughput**.
- **Memory utilization**
  U = live payload ÷ total memory managed ($0 < U \leq 1$); higher is better.
- **Different enemies**
  Latency suffers from search, locks and syscalls; **utilization** from fragmentation and metadata.
- **Design choices pull apart**
  Faster strategies often waste memory; **tighter packing** costs time.

Two metrics for an allocator: latency vs utilization

Speed of each request versus how efficiently memory is used.

1 Allocation Latency

How fast is one malloc/free?

What it is
- Time to serve one request (ns – µs)

Key metrics
- Mean and tail latency (p95, p99)
- Throughput (operations per second)

What affects it
- Searching for a free chunk
- Splitting / coalescing
- Lock contention
- System calls (brk, mmap)

Design goal
- Fast common case, no global locks

2 Memory Utilization

How much memory per byte used?

$$U = \frac{\text{Live payload}}{\text{Total memory managed}}$$

$0 < U \leq 1$ higher = better

What reduces it
- Internal / external fragmentation
- Metadata (headers, alignment)
- Free chunks kept, not returned to OS

Design goal
- Reuse and coalesce; return memory to the OS

Example: a 33 B request in a 48 B chunk gives U = 33 / 48 ≈ 0.69.

Live payload = bytes the app requested and still holds.

Trade-off Improving one metric often hurts the other: **balance low latency against high utilization.**

Key Takeaway

No single strategy is optimal for both low latency and high memory utilization; an allocator must balance them.

Overview of malloc() Chunk (64-bit ptmalloc)

Key Points

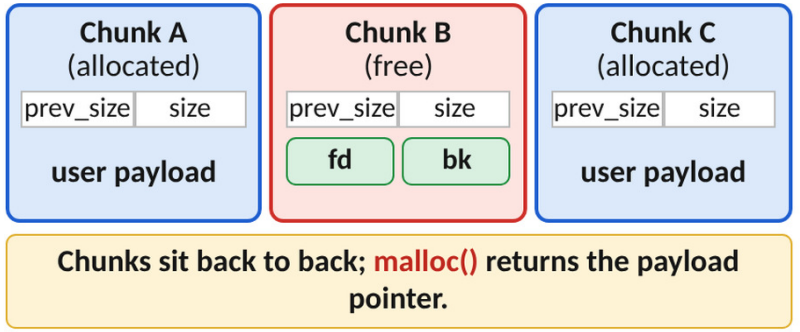
- **Header + payload**
  Allocated and free chunks share the same **header format**; a free chunk keeps **fd** (next) and **bk** (previous) free-list pointers in its payload.
- **Size field carries flags**
  The size is a **multiple of 16 B**, so its **low 3 bits** are free to hold the flags **P, M, A**.
- **prev_size only when P = 0**
  Used only if the **previous chunk is free**; otherwise it is part of the **previous chunk's payload**, so overhead is **8 B**.
- **mmap chunks are separate**
  M = 1 chunks are made by **malloc()** itself for large requests, not by a user mmap(); **free()** calls **munmap()**.

What does a malloc chunk look like in memory?

Header, payload and flag bits of a 64-bit glibc chunk.

Chunk layout in memory

Lower address → Higher address



Chunks sit back to back; malloc() returns the payload pointer.

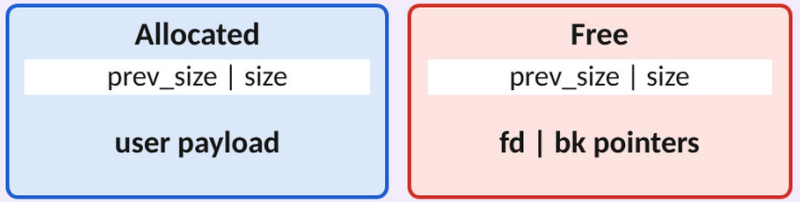
Size field (8 B)

chunk size (multiple of 16)	3	2	1	0
	0	A	M	P

Flag	Meaning
P PREV_INUSE	1: prev in use
M IS_MMAPPED	1: mmap chunk
A NON_MAIN_ARENA	1: not main arena

Low 3 bits are flags; size = size & ~0x7.

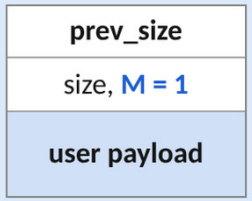
Allocated vs. free chunk (P bit)



P = 1: previous chunk is allocated, **prev_size is unused**.

P = 0: previous chunk is free, **prev_size holds its size**.

mmap chunk (M = 1)



- Created only when **malloc()** calls mmap() internally
- **free()** calls **munmap()** to release it
- A **direct mmap()** by the user creates **no chunk**; the OS manages it only as a **VMA**

Key Takeaway

Every chunk is a header plus payload; flag bits in the size field tell the allocator whether the neighbor is free, mmap-backed, or in another arena.

mmap Chunk

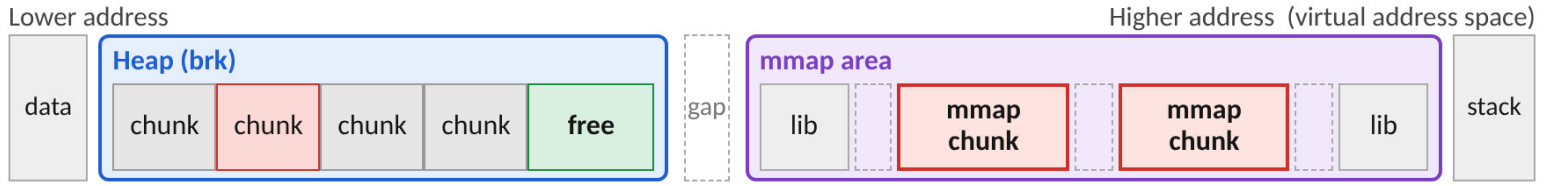
Key Points

- **Two places for chunks**
  Heap chunks sit back to back in the heap; **mmap chunks** live in the **mmap area**.
- **Request size decides**
  Below the threshold (default **128 KB**): heap chunk. **At or above**: **malloc() calls mmap()**.
- **Own mapping**
  The kernel creates a **VMA** (address range) and ptmalloc writes the header (**M = 1**, **prev_size** = offset); it has no neighbouring chunks.
- **Freed by munmap()**
  free() sees the M bit and unmaps the **whole mapping**: no coalescing, no reuse.

How is an mmap chunk allocated and freed?

Large requests get their own mapping instead of a spot in the heap.

Where each kind of chunk lives



n < threshold → heap chunk, **back to back** with its neighbours.

n ≥ threshold → **malloc() calls mmap()**: a chunk in its **own mapping**.

Heap chunk vs mmap chunk

	Heap chunk	mmap chunk
Created	n < threshold	n ≥ threshold
Lives in	Heap, with neighbours	Own mapping
Neighbours	Previous and next chunk	None
After free	Reused	munmap() to OS

Default threshold **128 KB** (M_MMAP_THRESHOLD, dynamic). A user's own **mmap()** makes only a VMA.

From the next slide on, "chunk" means a heap chunk unless stated otherwise.

Allocate and free

Allocate (n ≥ threshold)

kernel creates a VMA: mmap(NULL, size, ...)

▼

ptmalloc writes the header: size (M = 1), prev_size = offset

▼

return p = chunk start + header

Free: M = 1? then

munmap(mapping_start, mapping_size)

mapping_start = chunk_start − prev_size

Key Takeaway

Requests below the mmap threshold become heap chunks; larger ones become mmap chunks in their own mapping and are returned to the OS on free.

Minimum Chunk Size (64-bit ptmalloc)

Key Points

- **Start from n**
  The **requested size** passed to malloc().
- **Overhead and alignment**
  Add the **8 B overhead**, round up to **16 B**, and never go below **32 B**, the room a free chunk needs for **fd** and **bk**.
- **Usable size**
  usable = chunk − 8: the next chunk's prev_size is usable while this chunk is in use.
- **Chunk size picks the free list**
  malloc(1) to malloc(24) all use a **32 B** chunk; **malloc(25)** jumps to **48 B**.

How does malloc(n) become a chunk size?

Add the 8-byte overhead, align to 16 B, never go below 32 B.

1 Start from the request
n = bytes passed to malloc()

2 Add overhead and align
x = n + 8 (add the 8 B overhead)

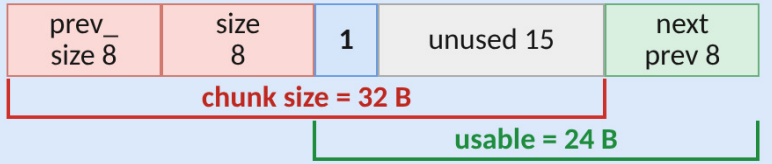
```
chunk = max(32, (x + 15) & ~15)
```

(x + 15) & ~15 = x rounded up to a multiple of 16

3 Resulting chunk size
At least 32 B; each size has its own free list

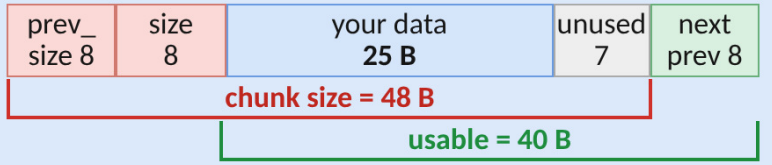
Example: malloc(1) vs malloc(25)

malloc(1)



1 + 8 = 9 → 16, but the **minimum is 32 B** (room for fd, bk).

malloc(25)



25 + 8 = 33 → **next multiple of 16 = 48**.

Request	n + 8, align 16	Chunk	Usable
`malloc(1)`	1 + 8 = 9 → 16 → 32	**32**	24
`malloc(24)`	24 + 8 = 32 → 32	**32**	24
`malloc(25)`	25 + 8 = 33 → 48	**48**	40
`malloc(62)`	62 + 8 = 70 → 80	**80**	72

Why steps of 16? Chunks start on **16 B boundaries** (alignment), so sizes are 32, 48, 64, … Usable = chunk − 8.

Key Takeaway

Each request becomes a 16-byte-aligned chunk of at least 32 B; chunks of the same size are kept on one free list for reuse.

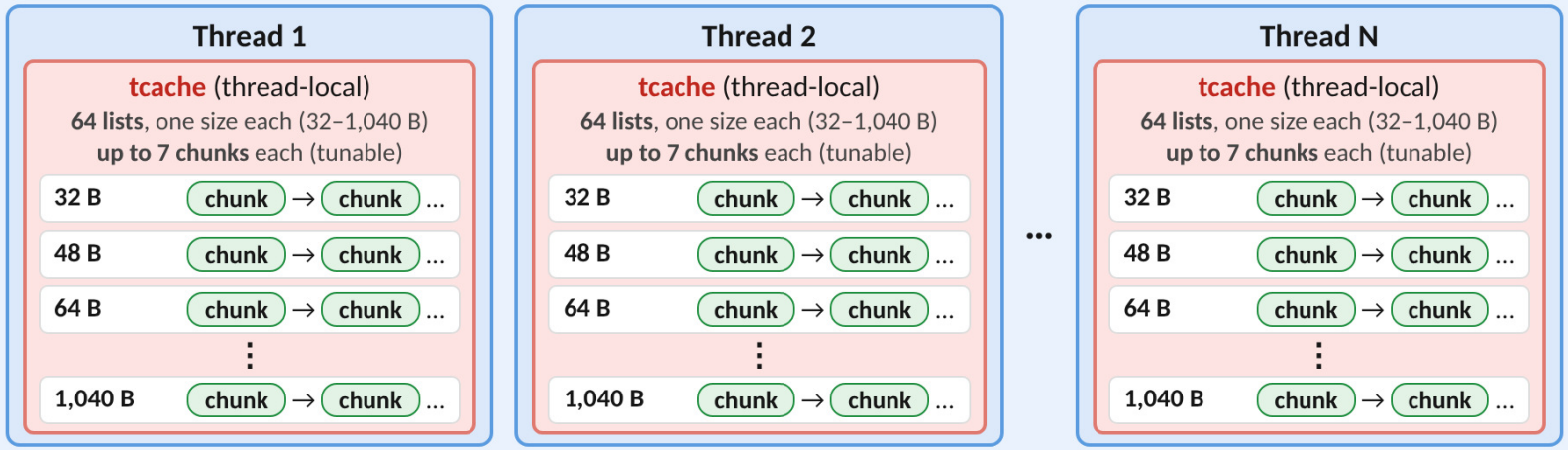
Overall Structure of Free Chunks in ptmalloc

Key Points

- **Free chunks are grouped by size**
  Freed chunks are kept in **lists per size**, so malloc() can reuse a matching one quickly.
- **tcache: one per thread**
  A **thread-local** cache, so it needs **no lock** and is checked **first**; each list holds up to **7 chunks** (tunable).
- **Arenas: shared by threads**
  On a tcache miss the thread uses an **arena**: the **main arena** plus **extra arenas** created on demand; access takes a **lock**.
- **Inside an arena**
  Fastbins, **unsorted bin**, **smallbins**, **largebins** hold free chunks by size; the **top chunk** is the free end of the heap, **not a bin**.

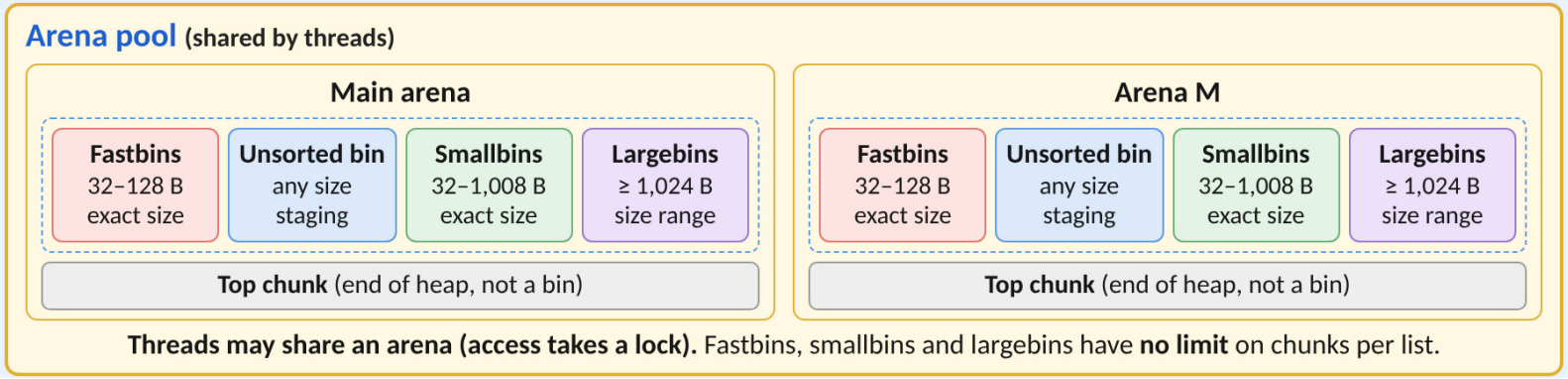
How do tcache and arenas organize free chunks?

Each thread owns a private cache; shared arenas hold the rest.



tcache limit: **7 chunks per list** by default; change it with `glibc.malloc.tcache_count`

▼ tcache miss: the thread uses an arena



Threads may share an arena (access takes a lock). Fastbins, smallbins and largebins have **no limit** on chunks per list.

Key Takeaway

A thread serves most requests from its lock-free tcache; only on a miss does it enter a shared arena (bins first, then the top chunk).

Details of Free Chunk Structure in ptmalloc

Key Points

- **Why so many lists?**
  Tuned for the **common case**: small, frequent requests get **fast reuse** (tcache, fastbins); larger, rarer ones accept **merging cost** to cut waste (small/largebins).
- **Why do sizes overlap?**
  They are **layers**: a freed chunk goes to **tcache** first (7 per list); when full it spills to a **fastbin** or the **unsorted bin**.
- **LIFO vs FIFO**
  tcache and fastbins reuse the most recently freed chunk (LIFO); **smallbins** reuse the oldest (FIFO).
- **Exact size vs. range**
  tcache, fastbins and smallbins hold **one exact size** per list; **largebins** hold a **size range** (best fit).

How do the ptmalloc structures differ?

Two levels of free chunk management: per-thread tcache and per-arena bins

Structure	Lists	Chunk size	Organization	Insert	Remove	Purpose
tcache per thread	64	32 – 1,040 B	singly linked	Head (LIFO)	Head (LIFO)	Exact-size lists, thread-local, no arena lock
fastbins	10 (7 used)	32 – 128 B	singly linked	Head (LIFO)	Head (LIFO)	Exact-size lists, fast, no immediate coalescing
unsorted bin	1	any size	doubly linked	Head	Search, reclassify	Staging: merged chunks wait here before sorting
smallbins	62	32 – 1,008 B	doubly linked	Head	Tail (FIFO)	Exact-size lists, coalesced when freed
largebins	63	≥ 1,024 B	doubly linked, sorted	Sorted insert	Best fit	Size-range lists, coalesced when freed
top chunk per arena	1	end of heap	single region	N/A	Split	Fallback when bins fail; not a bin

An arena holds the bins and a pointer to its top chunk; tcache sits outside it.

Key Takeaway

ptmalloc layers tcache and per-arena bins; each structure has its own list type and LIFO/FIFO policy, trading speed against memory use.

tcache Hit vs Miss

Key Points

- **Look up the exact size class**
  malloc(n) becomes a **chunk size**, then one **tcache bin** is checked.
- **Hit: fast path**
  Non-empty list: **pop the head**, no arena access, no lock, **O(1)**.
- **Miss: exact list is empty**
  tcache **never searches other classes** or splits larger cached chunks.
- **After a miss: arena path**
  Higher cost (**locking, search**); a **fastbin, smallbin or unsorted bin** hit may **refill** tcache.

What is the difference between a tcache hit and a miss?

Only the exact size class is checked; everything else goes to the arena.

1 Lookup in tcache

malloc(n) → chunk size → size class → check that bin

```
tcache[32 B] ▢▢...
tcache[48 B] ▢...
tcache[64 B] ▢▢▢...
```

2 HIT: fast path

The exact size list is **non-empty**.

A → B → C pop head → A returned

- No arena access, **no lock**
- **O(1)** head removal

3 MISS

The exact list is **empty**: other sizes are **not searched**, larger chunks are **not split**.

```
tcache[32 B] empty
tcache[48 B] ▢...
tcache[64 B] ▢▢▢...
```

4 After a miss: arena path

Fall back to the arena for a suitable chunk.

tcache miss → Select arena → Find in bins/top → Refill tcache → Return

- Slower than a hit: **lock and search**
- **Refill** before return, on a bin hit only

Refill after a miss (same size only, while the tcache list has room)

fastbin / smallbin hit	unsorted bin	largebin / top
return one; move the rest to tcache (3 chunks → 1 returned, 2 cached)	exact-size chunks are cached while scanning; one is returned	no refill

Key Takeaway

tcache gives very fast hits by reusing exact-size chunks per thread, and falls back to the arena on a miss.

Top Chunk

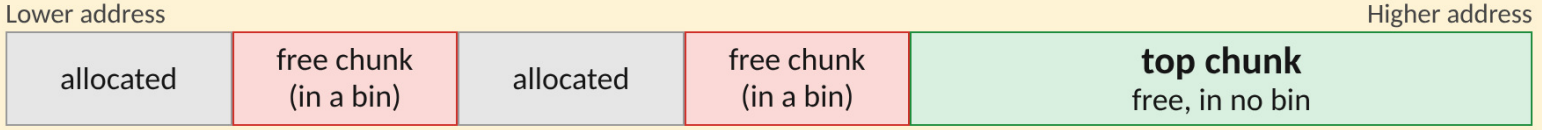
Key Points

- **Created at the first malloc**
  Not a bin: a free region at the **end of the heap**, made by **brk()** (about **132 KiB**) at the first malloc, when tcache metadata is allocated.
- **Checked on a miss**
  When tcache and the bins **miss**, the **request size** is compared with the **top chunk size**.
- **Split or grow**
  Fits: **cut** from the front, the rest stays top. Too big: **grow with brk()** (+ M_TOP_PAD margin), then split.
- **Merge and shrink**
  A free next to top **merges into it**; on a **free()**, a top of **M_TRIM_THRESHOLD** or more is **trimmed**, keeping a margin.

What is the top chunk?

The free region at the end of an arena's heap: how it is created, used, grown and returned.

Heap layout within an arena (conceptual)



1 Created at the first malloc

tcache metadata is allocated first (a small request), so **brk()** asks the OS for about **132 KiB** (that + a **128 KiB** margin, **M_TOP_PAD**). The unused part is the top chunk.

2 Checked on a miss

If tcache and every bin **miss**, malloc compares the **request size with the top chunk size**.

3 Request fits: split

Cut the needed chunk from the **front** of the top chunk; the rest stays as the **new top chunk**.

4 Request too big: grow

brk() adds memory (the shortfall + the **M_TOP_PAD** margin), then split. A request above **M_MMAP_THRESHOLD** gets an **mmap chunk** instead.

5 Merge when freed

A chunk freed **next to the top chunk** is merged into it (coalescing) instead of going to a bin.

6 Shrink on free

After a **free()**, if the top chunk is at least **M_TRIM_THRESHOLD**, **brk()** shrinks the heap and returns the excess to the OS, keeping an **M_TOP_PAD** margin.

Typical defaults: M_TOP_PAD 128 KiB, M_TRIM_THRESHOLD 128 KiB, M_MMAP_THRESHOLD 128 KB (dynamic). Values vary by glibc version.

Key Takeaway

The top chunk is created with the heap, split on demand, grown when short, and merged or trimmed as neighbours are freed.

malloc() Path for Small Chunks

Key Points

- **Exact-size lists first**
  tcache, **fastbin** and **smallbin** each hold chunks of one size, so a **hit** returns at once.
- **Refill on a hit**
  A fastbin or smallbin hit moves same-size chunks into **tcache** for the next call.
- **Unsorted bin: sort, not merge**
  The scan returns an **exact fit** or **sorts** chunks into bins; **no merging** happens here.
- **Then split, then grow**
  A larger bin or the **top chunk** is **split**; if none fits, the heap grows.

Small chunk (< 1,024 B): malloc() path

Chunk sizes (request + header, 16 B aligned). Follow the numbers: a miss moves to the next step.

1	**tcache**	Look in the tcache list of this exact size. **If found**, return it; **otherwise** go to step 2.	**hit → return**
2	**Fastbin** (≤ 128 B)	**If tcache missed** and size ≤ 128 B, search fastbin. **If found**, refill tcache and return; **otherwise** go to step 3.	**hit → return + refill**
3	**Smallbin** (< 1,024 B)	**If fastbin missed**, search smallbin. **If found**, refill tcache and return; **otherwise** go to step 4.	**hit → return + refill**
4	**Scan unsorted bin**	Scan from the **oldest chunk**. **If it fits exactly**, return it (no refill); **otherwise** move it to its matching smallbin or largebin (no merging) and go on. If none fits, step 5.	**exact fit → return**
5	**Scan next larger bin**	Search the bins **above the request size** (larger smallbins, then largebins) and take the **first non-empty** one; **split** its smallest chunk: return one part, the rest goes to unsorted. If none, step 6.	**split → return**
6	**Top chunk / OS**	**Split the top chunk.** If top is too small: **if fastbins hold chunks**, merge them and **retry from step 4**; **otherwise** grow the heap with brk().	**split or grow**

Where does merging happen? Not in this scan. Chunks are merged in **free()**, and when **fastbin chunks are consolidated** into the unsorted bin.

Key Takeaway

A small chunk request walks exact-size lists first; sorting, splitting and heap growth come only after misses.

Example: malloc() with Small Chunks

Key Points

- **Typical allocation path**
  A typical request tries **tcache**, then **fastbin / smallbin**, then the **unsorted bin**.
- **Refill**
  A fastbin hit moves the other same-size chunks to **tcache**, so the next call is a **hit**.
- **Unsorted: oldest first**
  Chunks enter at the **head** and are scanned from the **tail**; non-exact ones move to their bins.
- **Split a larger bin**
  If nothing fits, a **larger bin** is split and the **remainder** goes to unsorted.

Example: malloc() with small chunks

Chunk sizes in bytes. One thread, five requests in a row. Illustrative example using a simplified allocator state.

Starting state

`tcache[48]`	empty
`fastbin[48]`	48 48 48
`smallbin[112]`	112
`unsorted`	208 304
`largebin`	2,000

Unsorted list: new chunks enter at the head (left); the scan starts at the tail (right).

After the last step

`tcache[48]`	48
`fastbin[48]`	empty
`smallbin[112]`	empty
`smallbin[304]`	304
`unsorted`	1,488
`largebin`	empty

1	`malloc(40)` → `48 B`	tcache miss → **fastbin[48] hit**: return one, move the other **2 to tcache**.	**fastbin + refill**
2	`malloc(40)` → `48 B`	**tcache hit**: a refilled chunk.	**tcache hit**
3	`malloc(100)` → `112 B`	tcache and fastbin[112] miss → **smallbin[112] hit**.	**smallbin**
4	`malloc(200)` → `208 B`	Lists miss → scan unsorted from the tail: 304 B is not exact, **moved to smallbin**; then 208 B is an **exact fit**.	**unsorted: move + exact fit**
5	`malloc(500)` → `512 B`	Unsorted empty, no 512 B list → next non-empty larger bin (2,000 B): **split**; the **1,488 B remainder** goes to unsorted.	**larger bin + split**

Key Takeaway

A typical path checks tcache and size-specific bins before searching unsorted and larger bins; the actual path depends on chunk size and allocator state.

malloc() Path for Large Chunks

Key Points

- **Skip the small lists**
  Chunk sizes **≥ 1,024 B** have no fastbin or smallbin; **tcache** covers only up to **1,040 B**.
- **Consolidate first**
  Leftover **fastbin** chunks are **merged** and moved to the unsorted bin before searching.
- **Largebin: best fit**
  Largebins are sorted by size; the **smallest chunk that fits** is split and the **remainder** goes to unsorted.
- **Top chunk, then the OS**
  If no bin fits, split **top**; grow with **brk()**, or use **mmap()** for sizes ≥ threshold.

Large chunk (≥ 1,024 B): malloc() path

Chunk sizes (request + header). Follow the numbers: a miss moves to the next step.

1	**tcache**	**If** the chunk is ≤ 1,040 B and tcache has it, return it; **otherwise** go to step 2.	**hit → return**
2	**Fastbin / smallbin**	Chunk ≥ 1,024 B: these lists do not apply, so **skip** to step 3.	**skipped**
3	**Consolidate fastbins**	**If fastbins hold chunks**, merge them with free neighbours and move them to the unsorted bin. Then step 4.	**merge → unsorted**
4	**Scan unsorted bin**	Scan from the **oldest chunk**. **If it fits exactly**, return it; **otherwise** move it to its matching smallbin or largebin (no merging) and go on. If none fits, step 5.	**exact fit → return**
5	**Largebin best fit**	In the matching largebin (sorted large to small) take the **smallest chunk that fits**; split it, return one part, remainder to unsorted. If none, step 6.	**split → return**
6	**Larger bin, top chunk**	Try the **next non-empty larger bin** (next size up), then the **top chunk**, and split. If nothing fits, step 7.	**split → return**
7	**OS: mmap() or brk()**	**If** size ≥ **M_MMAP_THRESHOLD**, mmap() a separate chunk; **otherwise** grow the heap with brk() and retry.	**new memory**

Where does merging happen? Only in the **fastbin consolidation** step (and in free()). The unsorted scan and the largebin search only **sort and split**.

Key Takeaway

A large chunk request skips the small lists: consolidate, scan unsorted, take a largebin best fit, then the top chunk or the OS.

Example: malloc() with Large Chunks

Key Points

- **Skip, then consolidate**
  Fastbin and smallbin are **skipped**; leftover **fastbin** chunks are **merged** into unsorted.
- **Scan sorts the bins**
  The unsorted scan moves each chunk to its **smallbin** or **largebin**.
- **Best fit in largebin**
  The bin is sorted **large to small**; take the **smallest chunk that fits** and split it.
- **Top chunk, then mmap**
  If no bin fits, **split top**; a huge request gets an **mmap chunk**.

Example: malloc() with large chunks

Chunk sizes in bytes. One thread, three requests in a row.

Starting state

`fastbin[48]`	48 48 adjacent
`smallbin`	empty
`unsorted`	1,200
`largebin`	3,000 1,600
`top chunk`	about 100 KB

adjacent = the two 48 B chunks sit next to each other in memory, so they can merge. Largebin: sorted large (head, left) to small (tail, right).

After the last step

`fastbin[48]`	empty
`smallbin`	96 80
`unsorted`	empty
`largebin`	3,000 1,200
`top chunk`	about 96 KB
`mmap chunk`	1 (own mapping)

1 `malloc(1,500)` → `1,520 B`

a. Skip fastbin / smallbin.
b. Consolidate: fastbin 48 + 48 → 96 B, into unsorted.
c. Scan from the tail: 1,200 B → largebin; 96 B → smallbin.
d. Best fit: the smallest that fits is 1,600 B → **split**; the **80 B remainder** goes to unsorted.

consolidate + best fit

2 `malloc(3,500)` → `3,520 B`

Scan: 80 B → smallbin. No bin is big enough (largest is 3,000 B) → **split the top chunk**.

top chunk

3 `malloc(300,000)`

No bin fits, top is too small, size ≥ threshold → **malloc() calls mmap()**.

mmap chunk

Key Takeaway

A large chunk is served by consolidating, sorting through unsorted, a largebin best fit, then the top chunk or mmap.

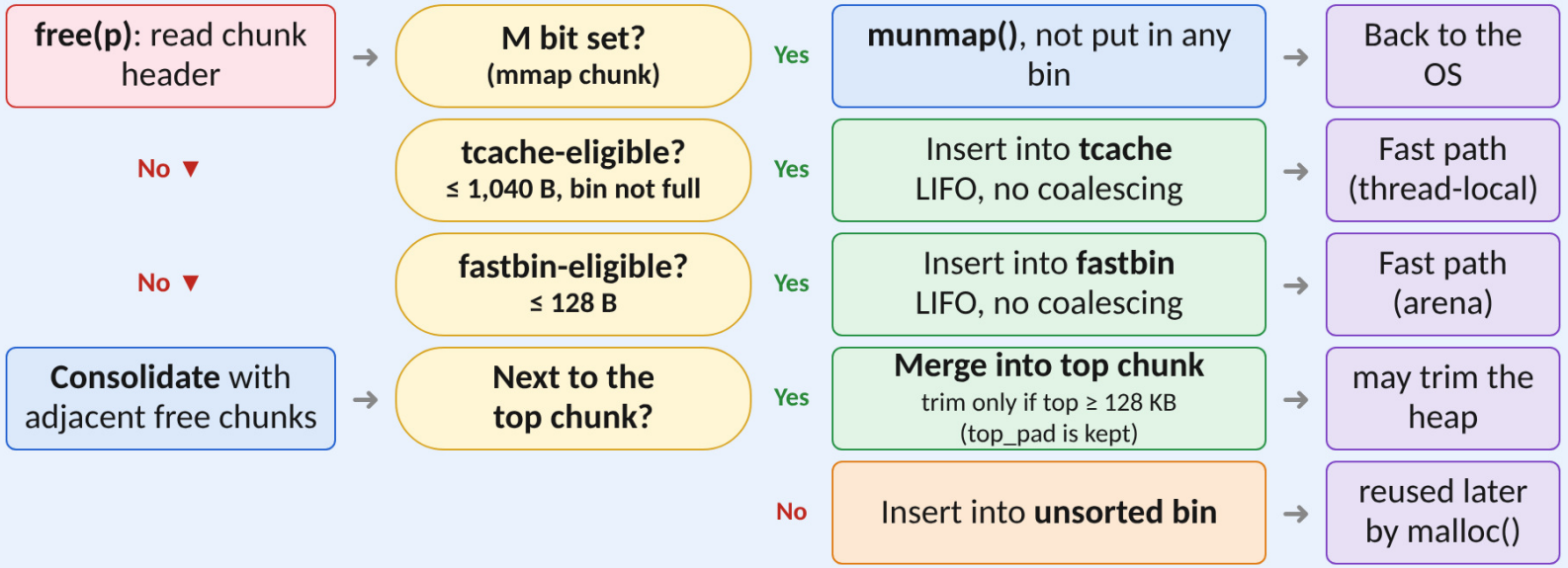
What Happens When free() is Called?

Key Points

- **The header decides the path**
  The **M bit** sends mmap chunks to **munmap()**; otherwise the **chunk size** picks the path.
- **Small chunks stay fast**
  Chunks up to **1,040 B** go to tcache (7 per size by default); up to **128 B** to fastbin; neither coalesces.
- **Larger chunks are coalesced**
  Merged with free neighbours; next to the top chunk they **join it**, otherwise they enter the **unsorted bin**.
- **Freed is not returned**
  Only **munmap()**, **heap trimming**, or **madvise()** give memory back to the OS.

What happens inside free()?

Simplified free path of a typical 64-bit glibc ptmalloc; the branch also depends on runtime state.

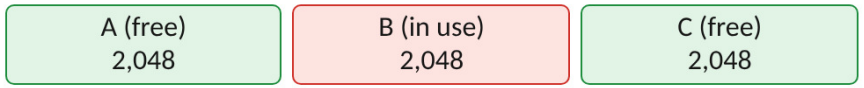


A **full tcache bin** (7 per size by default) sends the chunk to the next branch: size alone does not decide the path.

Chunk header flags

P PREV_INUSE: previous chunk in use
M IS_MMAPPED: mmap chunk
A NON_MAIN_ARENA: non-main arena

Coalescing example (bytes)



free(B) → **A + B + C (free) = 6,144**

Key Takeaway

Memory can return to the OS only through munmap(), heap trimming, or madvise(); small chunks stay in tcache/fastbin for fast reuse, larger ones are coalesced into bins or the top chunk.

Does free() Return Physical Memory?

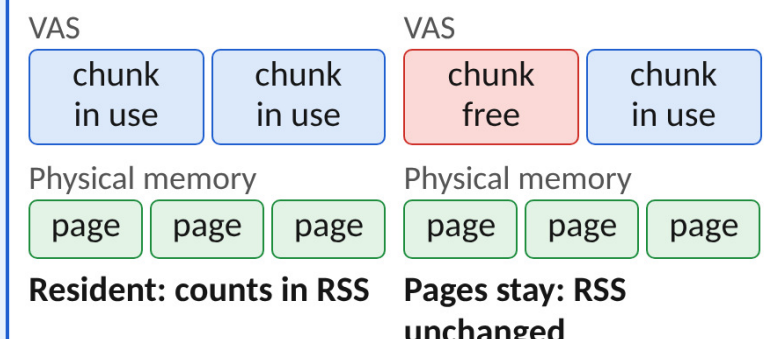
Key Points

- **It depends on the chunk type**
  A **heap chunk** stays with the allocator; an **mmap chunk** goes back to the OS.
- **Heap chunk (M = 0)**
  The heap mapping and its resident pages usually stay, so **RSS is unchanged**.
- **mmap chunk (M = 1)**
  free() calls **munmap()**: the VMA is removed, resident pages are released, **RSS drops**.
- **Heap memory can return later**
  Heap trimming (brk) or **madvise(MADV_DONTNEED)** can release pages and lower RSS; the VMA stays.

Does free() return physical memory?

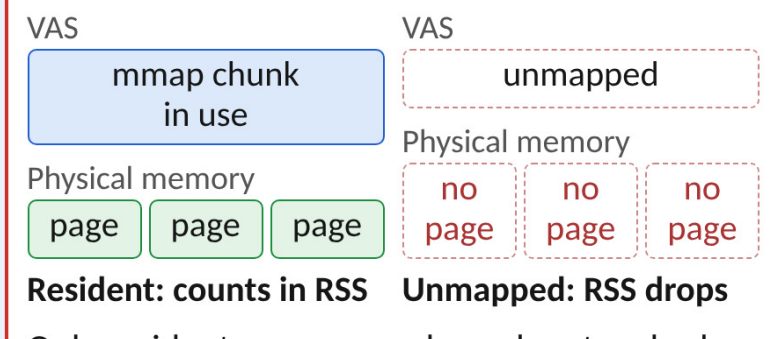
It depends on whether the chunk lives in the heap or in its own mmap region.

Heap chunk (M = 0): free()



Later, trimming (brk) or madvise can lower RSS; the VMA stays.

mmap chunk (M = 1): free() → munmap()



Only resident pages are released; untouched pages do not change RSS.

After free()	Heap chunk (M = 0)	mmap chunk (M = 1)
Returned to	The allocator (tcache, bins, top)	munmap() is called
VMA	Unchanged (heap stays mapped)	Removed
Back to the OS?	Later: brk trim or madvise	**Yes, immediately**

Key Takeaway

free() returns memory to the allocator, not to the OS; physical memory is released only when pages are unmapped (munmap) or explicitly reclaimed (heap trimming, madvise).

Observing RSS After free()

Key Points

- **Same allocation, different RSS**
  10,000 chunks of 4 KB are allocated and touched; **A** frees all of them, **B** frees every other one.
- **A: coalesce, then trim**
  All chunks merge into a **large top chunk**. At the **trim threshold** (128 KB), free() shrinks the heap with **brk()** but keeps **top_pad** (128 KB).
- **B: fragmented heap stays**
  Free chunks are separated by in-use chunks and the last chunk stays in use, so **there is no trimming**.
- **Observe with getchar()**
  Pause at each step and read **VmSize / VmRSS** from **/proc**: A falls, B stays. Freed chunks in B remain reusable.

Why does RSS stay high after free()?

Same allocation of 10,000 × 4 KB chunks; only the free pattern differs.

1 Test program (excerpt)

```
char *p[N];
printf("1. baseline\n"); getchar();
for (i = 0; i < N; i++) {
  p[i] = malloc(4096);
  memset(p[i], 1, 4096);
}
printf("2. after allocation\n");
getchar();
if (mode == 0)  // A: free all
  for (i=0; i<N; i++) free(p[i]);
else          // B: every other
  for (i=0; i<N; i+=2) free(p[i]);
printf("3. after free()\n");
getchar();
```

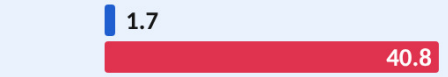
```
$ gcc -O2 -o rss_demo rss_demo.c
$ ./rss_demo 0   # A
$ ./rss_demo 1   # B
# at each pause, other terminal:
$ grep -E 'VmSize|VmRSS' \
    /proc/$(pidof rss_demo)/status
```

2 Measured values

Step	A: free all		B: every other	
	VmSize	RSS	VmSize	RSS
1. baseline	2.7	1.5	2.7	1.5
2. alloc.	41.9	40.8	41.9	40.8
3. free()	2.7	1.7	41.9	40.8

VmSize, VmRSS from /proc/PID/status (MB)
[heap] size (A): 132 KB → 40,272 KB → 140 KB
The first malloc() grows the heap by request + **top_pad** (128 KB), so the top chunk starts at about 128 KB.

RSS after free() (MB)



At step 3 (real output)

```
A: free all
VmSize:     2800 kB
VmRSS:      1760 kB
B: free every other
VmSize:    42932 kB
VmRSS:     41792 kB
```

3 Heap layout

A: free all

F = free chunk

The chunks merge into a big **top chunk**. If top ≥ **trim threshold** (128 KB), free() shrinks the heap with **brk()**, keeping **top_pad** (128 KB).

VmSize 41.9 → 2.7
RSS 40.8 → 1.7

B: every other

F free · U in use

5,000 chunks (≈ 20 MB) are free, but **isolated**: they never merge into top, so no trim.

VmSize 41.9 → 41.9
RSS 40.8 → 40.8

Key Takeaway

After free(), RSS depends on layout: freeing everything lets the top chunk shrink the heap, while freeing every other chunk leaves memory in the allocator and RSS stays high.

What is malloc_trim()?

Key Points

- **Targets physical memory**
  Touched pages are **physical memory**, and free lists keep them resident. **malloc_trim()** releases the free ones (**RSS ↓**).
- **Top chunk: shrink the heap**
  It can lower the heap end with **brk()**, so both **VmSize and RSS** decrease.
- **Interior free chunks: release pages**
  madvise(MADV_DONTNEED) frees fully free pages: **VmSize stays, RSS decreases**.
- **Only whole free pages**
  Page-aligned, fully free pages only; small free chunks may contain none.

What does malloc_trim() actually release?

The effect depends on where the free memory is.

malloc() + touch
allocates a **physical page**

→

free()
chunk stays in a free list: the **physical page is still resident**

→

malloc_trim()
releases **physical pages** (RSS ↓); **VAS is kept** except the heap end

A Free memory at the heap end brk()

Before malloc_trim()	After malloc_trim()
Allocated	Allocated
Top chunk (free)	returned to OS

The heap end moves down: **VmSize and RSS can both decrease.**

B Free memory in the middle madvise

Before malloc_trim()	After malloc_trim()
Allocated	Allocated
Free chunk (middle)	Free chunk (pages released)
Allocated	Allocated

VMA unchanged, free pages released: **VmSize same, RSS can drop.**

malloc_trim() targets physical memory, not the address space.
Released pages stay reusable: they are supplied again on the next access (page fault).

Key Takeaway

malloc_trim() reduces the physical memory (RSS) held by free chunks; the virtual address space is kept, except at the heap end.

Implications in Memory-Constrained Systems

Key Points

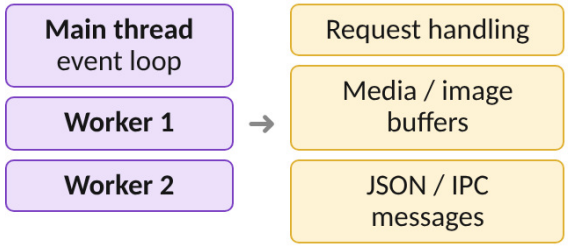
- **High RSS is not necessarily a leak**
  After a workload, live objects fall but RSS often stays high: **allocator retention** and **fragmentation**.
- **glibc keeps freed memory**
  Chunks in bins, partly used pages, and free memory spread over **arenas** are all retained.
- **Costly on embedded devices**
  On **memory-constrained** systems, retaining unused pages is expensive.
- **Measure, then trim selectively**
  Compare RSS with app-level metrics (**/proc, smaps, heap profiling**); call **malloc_trim()** after large temporary workloads or at idle, not frequently.

Why does RSS stay high in an embedded service?

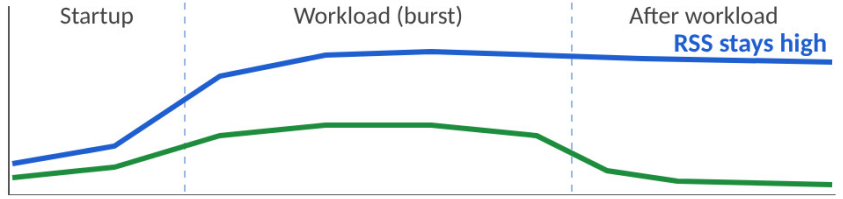
Many objects are freed after a workload, yet resident memory barely drops.

1 Long-running embedded service

A native process with an event loop and worker threads



2 Memory usage over time (illustrative)



3 Why does RSS stay high in this case?

1 Freed chunks stay in the allocator

- free() returns chunks to **tcache, fastbin, bins**
- They stay available for reuse
- No immediate system call

2 Partial page occupancy

- A page is reclaimable only if **all of it** is free
- One live allocation **pins the whole page**
- Small free chunks may span no full page

3 Multiple arenas

- Each thread has a tcache; threads may **share an arena**
- Free memory is **spread across arenas**
- Cross-arena coalescing is not possible

4 Allocator design trade-off

- Keeping freed memory **lowers allocation latency**
- It also avoids system-call overhead
- The cost is a **larger footprint (RSS)**

Key Takeaway

High RSS after a workload is often allocator retention, not a leak; on memory-constrained devices, measure first and call malloc_trim() selectively.

Checkpoint: Inside the Allocator

Key Points

- **Chunk = header + payload**
  malloc() returns the **payload**; sizes round to **16 B**, minimum **32 B**.
- **Fragmentation costs memory**
  Internal waste is inside a chunk; **external** waste is scattered free blocks.
- **tcache first, then the arena**
  A **hit** is lock-free; a **miss** goes to bins, the top chunk, then the OS.
- **free() is not a release**
  Memory goes back to the **allocator**; only **munmap**, **trim** or **madvise** return pages.

Checkpoint: Inside the Allocator

Try to answer each question first, then check the answer.

1 What is a chunk, and what does malloc() return?

A. The allocator's unit: **header + payload**. malloc() returns a pointer to the **payload**.

2 How big is the chunk for malloc(33) on 64-bit glibc?

A. 33 + 8 = 41, rounded up to a multiple of 16: **48 B** (minimum 32 B).

3 Internal vs external fragmentation?

A. **Internal:** waste inside a chunk. **External:** free memory split into small separate blocks.

4 What happens on a tcache hit and on a miss?

A. **Hit:** pop the head, no lock. **Miss:** arena path with a lock: bins, top chunk, then the OS.

5 Why does RSS often stay high after free()?

A. free() returns chunks to the **allocator**, not the OS; the heap stays mapped.

6 What can malloc_trim() release?

A. The **top of the heap** via brk() and fully free pages via **madvise()**; VmSize may stay.

Key Takeaway

Can you follow a request through ptmalloc and explain why RSS often stays high after free()?

Week 7 Summary

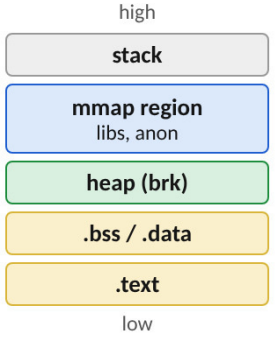
Key Points

- **Each process has its own VAS**
  It is created from the ELF and managed with VMAs.
- **brk() and mmap() grow it**
  They extend the address space for the heap and for large allocations.
- **glibc malloc builds on them**
  It uses brk and mmap with arenas, tcache (fastbin, smallbin, largebin, ...).
- **Release vs. reclaim**
  Allocator **releases** (munmap, trim, madvise); the kernel **reclaims** under pressure (swap, OOM).

Week 7 at a glance

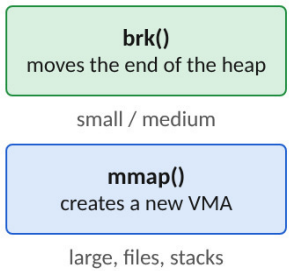
From the ELF file to the allocator, and back to the OS.

1 Process view ELF → VAS → VMAs



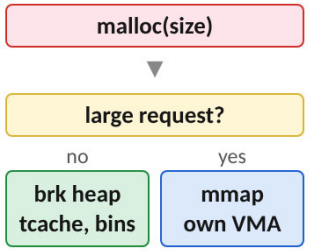
- The kernel reads the **ELF** headers and creates **VMAs**.
- Each VMA has its own **permissions** (R/W/X).
- **ASLR** randomizes base addresses.

2 Growing the VAS brk() and mmap()



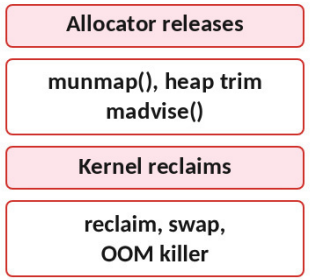
- **brk()** extends the heap: small and medium requests.
- **mmap()** adds a VMA: large requests, files, stacks.
- The process asks; the **kernel** adds or extends VMAs.

3 malloc (glibc) uses brk and mmap



- Small and medium: **brk()** plus tcache and bins.
- Large (default ≥ 128 KB): **mmap()**.
- Multiple **arenas** reduce lock contention.

4 Release vs. reclaim allocator vs. kernel



- **malloc_trim()** shrinks the heap top and releases free pages.
- **munmap()** releases an mmap chunk at once.
- Under pressure the kernel reclaims, swaps, or kills (**OOM**).

Key Takeaway

A process has its own virtual address space; malloc grows it with brk() and mmap(); the allocator releases memory (munmap, trim, madvise), and the kernel reclaims it under pressure (swap, OOM killer).

End of Week 7

Sungyong Park, Ph.D. | **parksy@sogang.ac.kr**